Procedural World Generation, Spatial Epidemic Simulation and Real-Time Rendering for a Browser Strategy Game

Algorithms, Mathematical Models and Performance Engineering

Kazak Vladimir

Technical Report

November 2026

Abstract

This report describes the technical design of a real-time strategy game in which a spreading organism colonises a procedurally generated planet while the planet's societies detect it and race to stop it. We present (i) a deterministic world-generation pipeline that turns a single integer seed into continents, regions, climates and a multimodal transport network, combining value noise, fractional Brownian motion, order statistics, connected-component labelling, farthest-point sampling, a noisy multi-source Dijkstra partition, union–find connectivity repair and weighted A* routing; (ii) a discrete-time stochastic metapopulation model with logistic growth inside regions, link-based transmission, an endogenous detection and countermeasure process with adaptive difficulty, and containment behaviour; (iii) an environment model with hemispheric seasons, weather fronts treated as a marked point process with physically motivated corrections, and a ground state that integrates the weather through first-order relaxation equations; (iv) a sub-regional growth model that derives the order in which a region is covered from a multi-source geodesic arrival time on the terrain and equalises it by rank, so that the displayed area is an exact function of the simulated coverage; (v) a rendering system built from analytic, anti-aliased procedural shading: smooth region borders from a B-spline vote over a cell partition, seasonal and weather-driven land shading, and a water surface with analytic parallax and refraction that needs no copy of the scene; and (vi) the performance engineering that keeps generation, simulation and rendering within a browser's frame budget. The methods are illustrated with the output of the implementation and validated by determinism checks, headless simulated players, offline evaluation of the shading mathematics on the painted map, and runtime measurements.

Keywords: procedural content generation; value noise; fractional Brownian motion; farthest-point sampling; Dijkstra partition; geodesic distance; rank equalisation; A* search; logistic growth; metapopulation model; marked point process; relaxation dynamics; B-spline filtering; analytic anti-aliasing; parallax and refraction; stylised rendering; automated playtesting; real-time systems.

Contents

1 Introduction **4**
1.1 Problem and objectives . . . 4
1.2 Structure of the report . . . 4

2 System architecture **4**

3 Procedural world generation **5**
3.1 Deterministic pseudo-random source . . . 5
3.2 Value noise and fractional Brownian motion . . . 5
3.3 Domain shaping . . . 6
3.4 Sea level as an order statistic . . . 7
3.5 Landmass extraction . . . 8
3.6 Region seeding by farthest-point sampling . . . 8
3.7 Region partition by noisy multi-source Dijkstra . . . 9
3.8 Region attributes . . . 9
3.9 Transport network . . . 10
3.10 Maritime routing . . . 11
3.11 Determinism and incremental evaluation . . . 12

4 Epidemic simulation **12**
4.1 Time and pacing . . . 12
4.2 Growth inside a region . . . 13
4.3 Transmission between regions . . . 14
4.4 Sub-regional growth order . . . 14
4.5 Detection and countermeasure . . . 16
4.6 Containment: closures and laboratories . . . 17
4.7 Endgame . . . 18
4.8 Player economy . . . 18
4.9 Outcomes . . . 19

5 Environment model: seasons, weather and ground memory **19**
5.1 Seasonal clock and hemispheres . . . 19
5.2 Weather fronts as a marked point process . . . 20
5.3 Ground state: the memory of the weather . . . 21

6 Rendering **22**
6.1 Base map . . . 22
6.2 GPU colony shader . . . 24
6.3 Seasons and ground state on the land . . . 24
6.4 Water . . . 25
6.5 Weather, night and light . . . 27

7 Performance engineering **27**
7.1 Methodology . . . 27
7.2 Time-sliced generation . . . 27
7.3 Linear-time selection . . . 28
7.4 Tiled texture uploads . . . 28
7.5 Cost of each subsystem . . . 28
7.6 Further measures . . . 29
7.7 Delivery size . . . 29

8 Validation **30**
8.1 Determinism . . . 30
8.2 Balance with simulated players . . . 30
8.3 Statistics of the environment . . . 31
8.4 Offline evaluation of procedural shading . . . 31
8.5 Runtime . . . 31

9 Discussion and limitations **31**

10 Conclusion **32**

A Symbols **33**

B Supplementary animations **35**

1 Introduction

1.1 Problem and objectives

Games about the spread of an infection over a world map, while an opposing force develops countermeasures, combine three technical problems: the world on which the spread happens, the dynamics of the spread and the response, and a presentation that makes both legible in real time. The system described here targets short sessions in a web browser and pursues three objectives:

1. **Replayability without authored content.** Every session takes place on a new world generated from a seed; the generator must produce varied yet always playable worlds.
2. **Legible, deterministic rules.** The simulation must be simple enough to reason about, deterministic for a given seed and sequence of player decisions, and cheap enough to be run many thousands of times for automated balancing.
3. **Fluid presentation within tight budgets.** Generation, simulation and rendering must fit a real-time frame budget on modest hardware, and the delivered application must fit a small download budget.

1.2 Structure of the report

Section 2 outlines the architecture. Section 3 presents world generation, Section 4 the simulation and the sub-regional growth order, Section 5 the environment model (seasons, weather, ground state), Section 6 the rendering, and Section 7 the performance engineering. Section 8 describes how the system is validated; Section 9 discusses limitations. A table of symbols is given in Appendix A, and Appendix B contains animations of the generation and the simulation.

2 System architecture

The system is implemented in C# on the Unity engine and delivered to browsers through WebGL. Its code is split into three layers with one-way dependencies (Fig. 1). The *world generator* is a pure function of the seed. The *rules* hold the state of one session and advance it by whole simulated days through a single entry point; they use plain numeric data only and no engine objects. The *presentation* (map views, interface screens and GPU shaders) reads that state and draws it. Because the two lower layers are independent of the engine, the same rules run inside the application and in a command-line harness used for automated balancing and measurement.

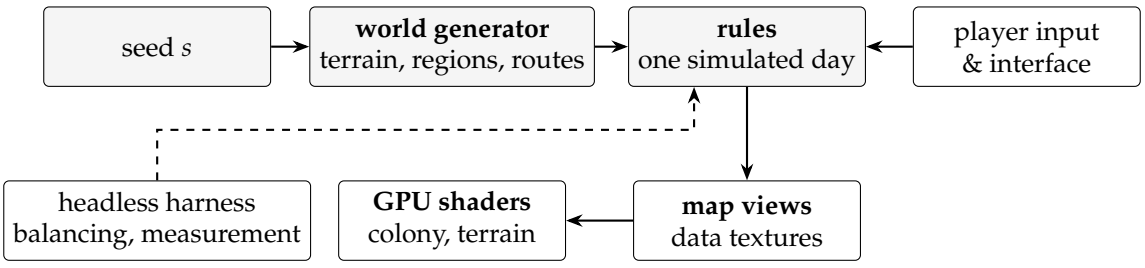


Figure 1: Layered architecture. Solid arrows: data flow during play; dashed: the same rules driven by a command-line harness without the engine.

The world is a regular grid of $W \times H$ cells; cell c has integer coordinates (x_c, y_c) and index $c = y_c W + x_c$. Two grid resolutions are supported, a standard one and a larger one for an extended mode, with $N = WH$ cells in total.

3 Procedural world generation

The generator executes the pipeline of Algorithm 1. All random decisions of the generator draw from one deterministic stream, so the output is a function of the seed alone.

Algorithm 1: World generation pipeline

Input: seed s, grid resolution
Output: world: heights, land mask, regions, attributes, transport network

1 initialise the pseudo-random stream from s; draw a geography and a society preset;
2 compute the height field from fractional Brownian motion with edge attenuation (Sections 3.2 and 3.3);
3 $\theta \leftarrow$ the order statistic leaving the target land share above it; land $\leftarrow h > \theta$ (Section 3.4);
4 label connected landmasses and keep the significant ones (Section 3.5);
5 place region centres by farthest-point sampling (Section 3.6);
6 partition land by multi-source Dijkstra with noisy costs (Section 3.7);
7 assign region attributes (Section 3.8);
8 build land, sea and air links and repair connectivity with union–find (Section 3.9);
9 route sea lanes with weighted A* and smooth them (Section 3.10);

3.1 Deterministic pseudo-random source

Randomness is supplied by *mulberry32*, a 32-bit counter-based generator. With state a and arithmetic modulo 2^{32},

$$\begin{aligned} a &\leftarrow a + \texttt{0x6D2B79F5}, \\ t &\leftarrow \big(a \oplus (a \gg 15)\big) \cdot (a \mid 1), \\ t &\leftarrow \Big(t + \big(t \oplus (t \gg 7)\big) \cdot (t \mid 61)\Big) \oplus t, \\ u &= \big(t \oplus (t \gg 14)\big)/2^{32} \in [0,1). \end{aligned} \tag{1}$$

Because the increment is odd, the state visits all 2^{32} values before repeating. The generator is fast and fully portable, so the same seed yields the same world on every platform. Figure 2 shows a sample of its output: the histogram is flat (a χ^2 test over equal bins is consistent with uniformity) and successive pairs fill the unit square without visible structure.

3.2 Value noise and fractional Brownian motion

Terrain is built from *value noise* [1, 2]. A table P of random values is filled from the stream, and integer lattice points are hashed into it with two large odd multipliers μ_1, μ_2:

$$L(i,j) = P\big[(\mu_1 i \oplus \mu_2 j) \bmod |P|\big]. \tag{2}$$

Between lattice points, the four surrounding values $a = L(i,j)$, $b = L(i{+}1,j)$, $c = L(i,j{+}1)$, $d = L(i{+}1,j{+}1)$ are blended with the smoothstep weights $s(t) = t^2(3-2t)$. Their derivative vanishes at $t \in \{0,1\}$, so the surface has no creases along lattice lines:

$$N(x,y) = a + (b-a)\,s_x + (c-a)\,s_y + (a-b-c+d)\,s_x s_y, \qquad s_x = s(x - \lfloor x \rfloor),\ s_y = s(y - \lfloor y \rfloor). \tag{3}$$

A single layer produces smooth, blob-like relief (Fig. 3). Natural coastlines are statistically self-similar [3], so several octaves are summed with doubling frequency and halving amplitude,

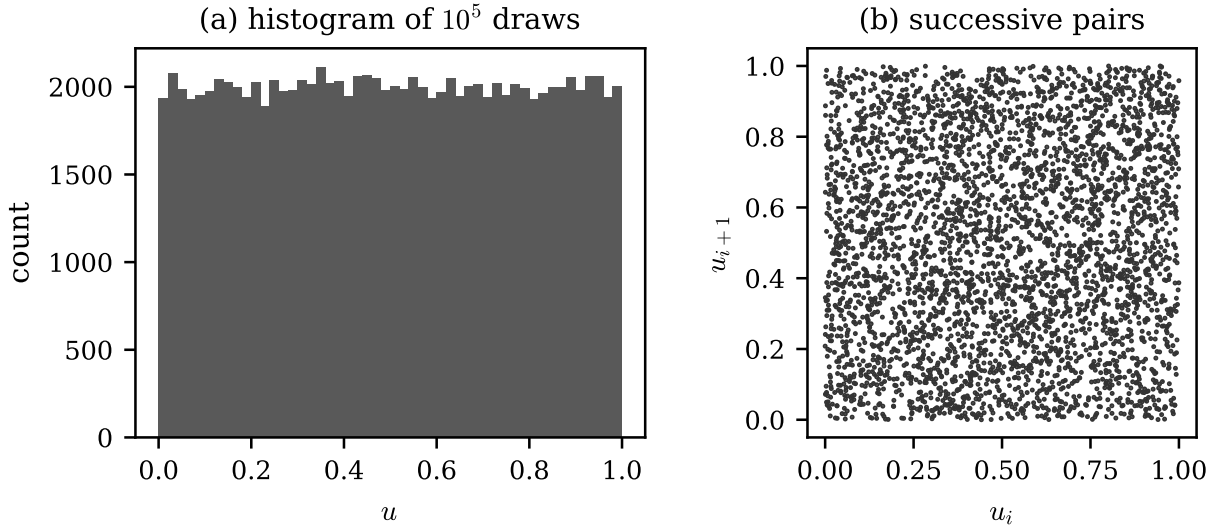


Figure 2: Output of the generator. (a) Distribution of 10^5 values. (b) Successive pairs (u_i, u_{i+1}).

a fractional Brownian motion (fBm) [4]:

$$F(x,y) = \frac{\sum_{o=0}^{O-1} 2^{-(o+1)}\, N(2^o x, 2^o y)}{\sum_{o=0}^{O-1} 2^{-(o+1)}}. \tag{4}$$

The normalisation keeps $F \in [0,1]$. Low octaves set the continents, high octaves add bays and capes (Fig. 4).

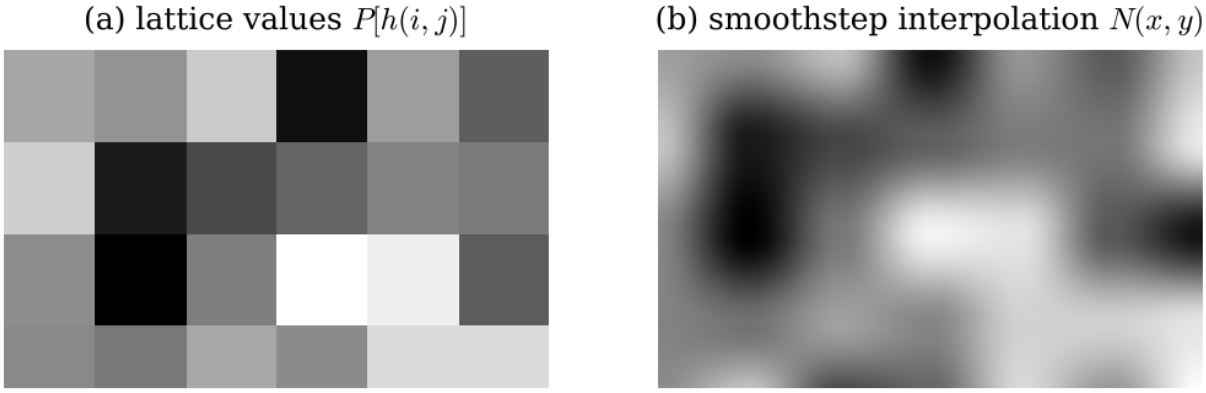


Figure 3: Value noise. (a) Random lattice values. (b) The same values blended with Eq. (3).

3.3 Domain shaping

The height of a cell samples the fBm in a domain scaled by a frequency ϕ, stretched horizontally by an aspect factor α and offset by a random vector (o_x, o_y), then corrected near the map border and, optionally, raised towards the centre:

$$h(x,y) = F\Big(\alpha\phi\,\tfrac{x}{W} + o_x,\ \phi\,\tfrac{y}{H} + o_y\Big) - \kappa\,(1-e)^2 + \gamma\big(1 - \lambda_c\, r\big), \tag{5}$$

where r is the normalised distance from the centre and $e \in [0,1]$ the edge proximity

$$e = \min\Big(1,\ \min\big(\tfrac{\min(x, W-1-x)}{m_x W},\ \tfrac{\min(y, H-1-y)}{m_y H}\big)\Big) \tag{6}$$

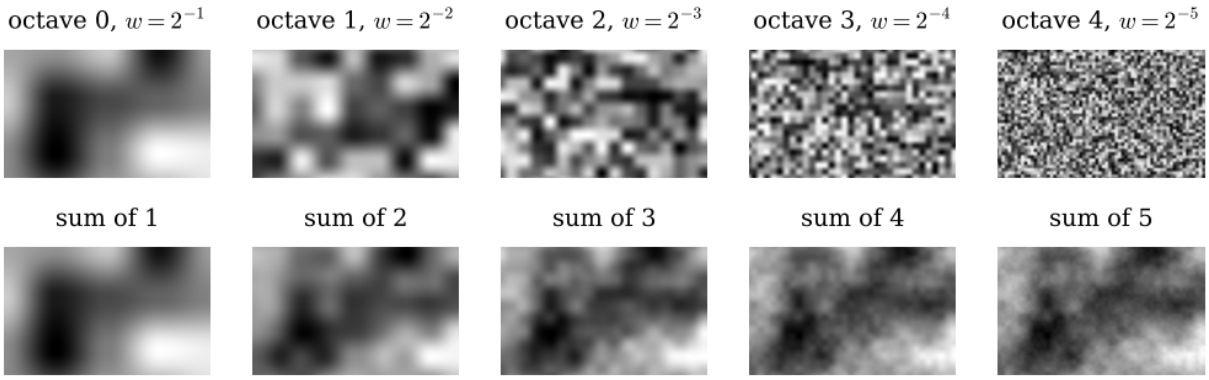


Figure 4: Fractional Brownian motion. Top: individual octaves with weights $2^{-(o+1)}$. Bottom: partial sums of Eq. (4).

with relative margins m_x, m_y. The quadratic attenuation leaves the interior untouched and depresses only the margins, steeply at the border, so every world is surrounded by navigable sea (Fig. 5). The centre term γ is used by presets that favour a single supercontinent. Larger grids scale ϕ with the grid size, which keeps the size of individual features constant while their number grows.

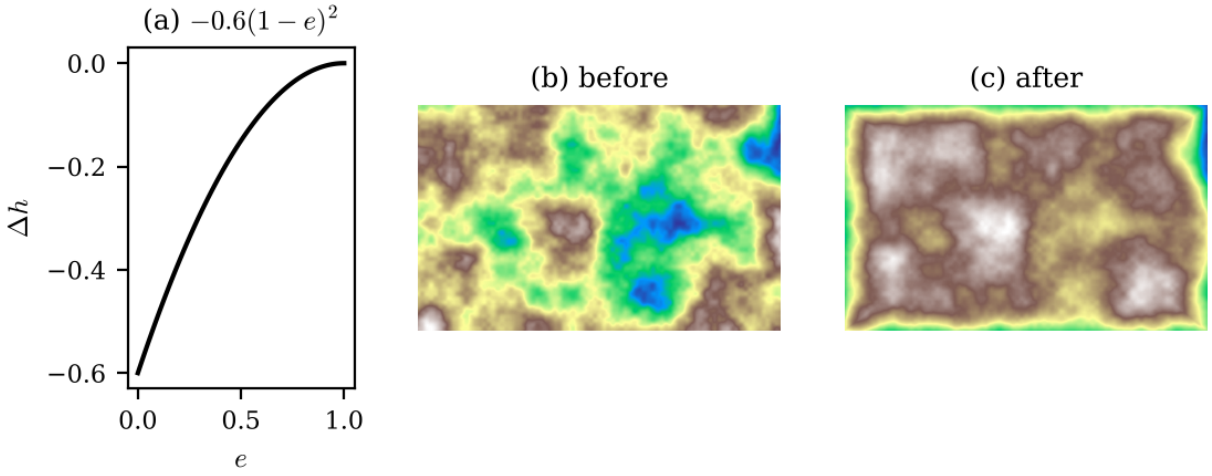


Figure 5: Edge attenuation. (a) Profile of the correction. (b,c) A height field before and after.

3.4 Sea level as an order statistic

Instead of fixing a water level, the generator fixes the land share λ requested by the geography preset and solves for the level. The sea level is the order statistic

$$\theta = h_{(k)}, \qquad k = \lfloor N(1-\lambda) \rfloor, \tag{7}$$

so that exactly a fraction λ of the cells lies above it (Fig. 6). Sorting would cost $O(N \log N)$. The implementation uses Hoare's selection algorithm [5] (quickselect), which partitions around a pivot and recurses only into the side that contains position k, giving expected $O(N)$ time and the identical result. Border cells are forced to water.

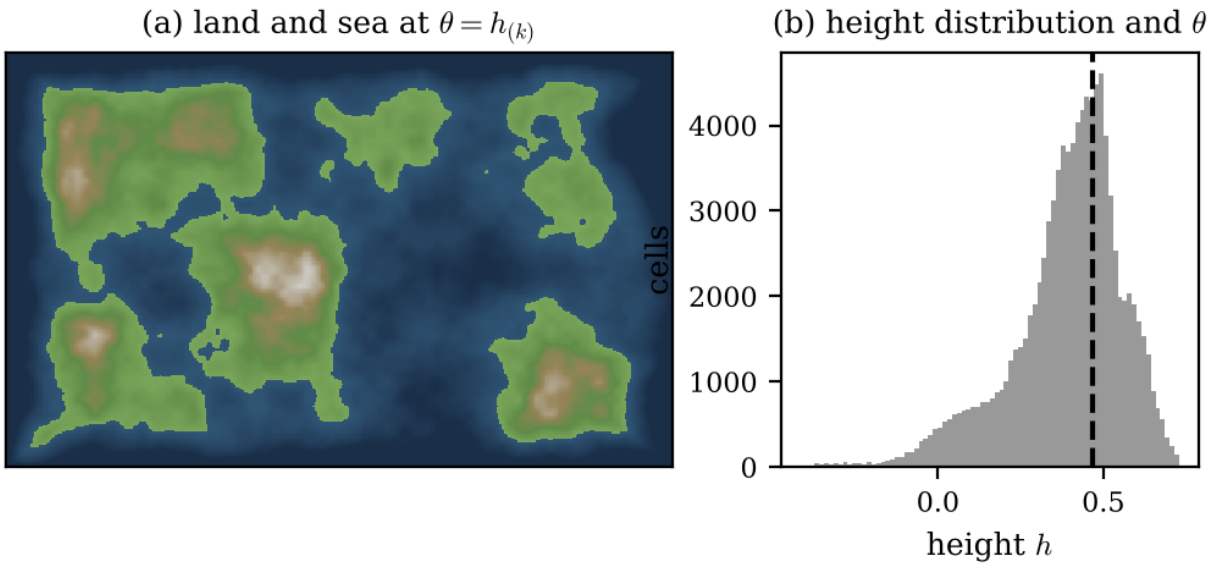


Figure 6: Sea level. (a) Resulting land and sea. (b) Height distribution with the threshold θ.

3.5 Landmass extraction

Connected components of the land mask under 4-connectivity are labelled by breadth-first search [6]: each unvisited land cell seeds a queue that floods its component, in $O(N)$ time overall. Components are sorted by size, and only the largest ones above a minimum size are kept (the largest is always kept), so that no region is created on an insignificant islet (Fig. 7).



Figure 7: Landmasses labelled by breadth-first search. Grey: components discarded by the size filter.

3.6 Region seeding by farthest-point sampling

The land is divided into N_R regions, with N_R drawn from a range that grows with the grid resolution. Landmass C receives $n_C = \max\big(1, \operatorname{round}(N_R|C|/|L|)\big)$ centres, $|L|$ being the total land area. Within each landmass the centres are placed by *farthest-point sampling* [7, 8]: starting from a random cell, each new centre is the cell that maximises the distance to all previous centres,

$$s_{k+1} = \arg\max_{p \in C} \min_{j \le k} \|p - s_j\|^2. \tag{8}$$

The inner minimum is maintained incrementally (Algorithm 2), so each step is a single pass over C and the total cost is $O(n_C|C|)$. The greedy rule is within a factor of two of the optimal covering radius [7]. It yields well-spread but irregular centres (Fig. 8).

Algorithm 2: Farthest-point sampling inside one landmass

$D[p] \leftarrow \infty$ for all $p \in C$; $s \leftarrow$ random cell of C;
for $k \leftarrow 1$ **to** n_C **do**
  emit centre s;
  foreach $p \in C$ **do** $D[p] \leftarrow \min\big(D[p], \|p - s\|^2\big)$;
  $s \leftarrow \arg\max_{p \in C} D[p]$;

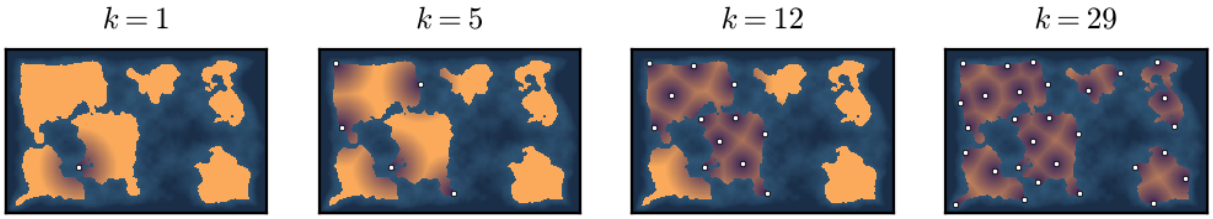


Figure 8: Farthest-point sampling after k centres. Brightness: distance to the nearest centre; the next centre is placed at the brightest cell of its landmass.

3.7 Region partition by noisy multi-source Dijkstra

Every land cell is assigned to the centre with the smallest *geodesic* cost, computed for all centres at once by a multi-source Dijkstra search [9] with a binary heap. Entering cell q costs

$$w(q) = 1 + \beta_w \, N(\nu x_q + o_x, \; \nu y_q + o_y), \tag{9}$$

with a low noise frequency ν and weight β_w. The region of q is $\arg\min_k d_k(q)$, where d_k is the cheapest path cost from centre k over land. With constant w the partition reduces to a discrete Voronoi diagram [10] with straight bisector borders. The noise term makes some ground more expensive than other ground, so borders follow valleys of the noise and acquire the meanders of natural boundaries (Fig. 10). Each cell is settled once, giving $O(N \log N)$. Figure 9 shows the growing fronts (Animation S1).

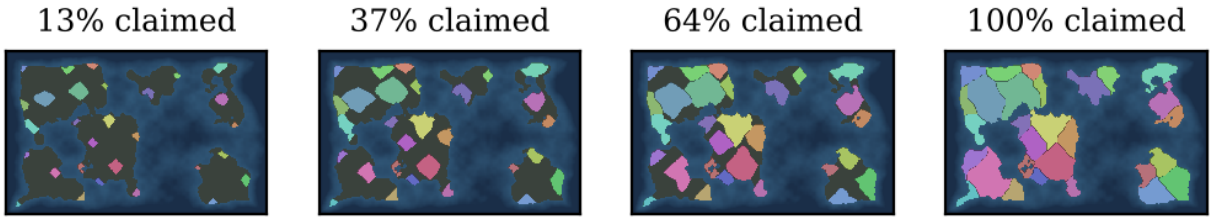


Figure 9: Multi-source Dijkstra growth at four stages (Animation S1 in Appendix B).

3.8 Region attributes

For each region the generator computes the area A, the centroid $(\bar{x}, \bar{y})$ and a label cell (the cell closest to the centroid), and draws its attributes:

(a) Euclidean Voronoi



(b) noisy-cost geodesic growth



Figure 10: Same centres, two partitions. (a) Nearest centre in the Euclidean metric. (b) Noisy geodesic metric, Eq. (9).

- **Climate** from latitude, $\ell = 2\,|\bar{y}/H - \frac{1}{2}|$ plus a small random jitter: cold above a threshold τ_c, hot below τ_h, temperate in between (Fig. 11). A repair pass then removes impossible neighbours: wherever a hot region borders a cold one, the one whose latitude is closer to the temperate band becomes temperate, so that a hot and a cold region never share a border.
- **Wealth** (rich, middle, poor) and **density** (dense or sparse), drawn from categorical distributions.
- **Population** proportional to area, scaled by density, by a random factor, and reduced in cold regions.
- **Paranoia** π, the level of global spread at which the region begins to close its borders (Section 4.6), drawn uniformly and lowered for rich and isolated regions.

A world-level *society* preset modulates several of these quantities globally, for example flight density, detection speed or research speed, so that worlds differ in character as well as in shape.

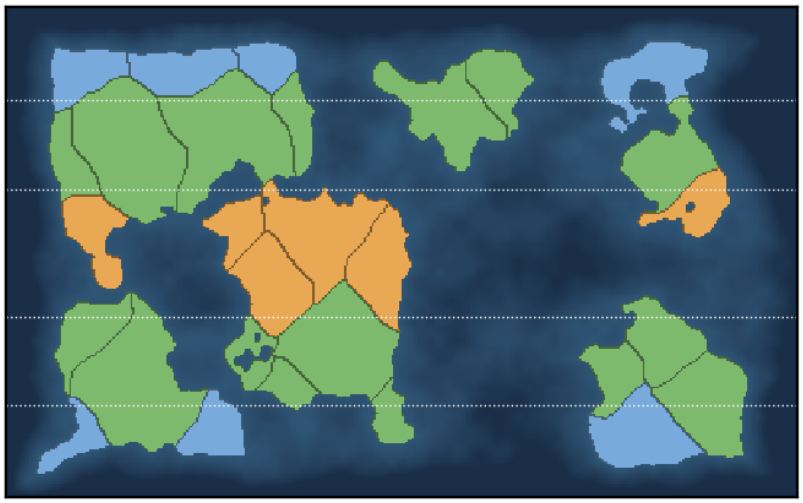


Figure 11: Climate zones (blue cold, green temperate, orange hot) with the latitude thresholds τ_c, τ_h (dotted).

3.9 Transport network

Three kinds of link form a multigraph over the regions (Fig. 12):

- **Land links** join every pair of regions that share a border.
- **Sea links.** Regions touching the *open* ocean receive a port with some probability. Open ocean is the water connected to the map edge, found by a flood fill from the border, so inland lakes are excluded. Every landmass with a coast is guaranteed at least one port, and each port is linked to its nearest ports that are not land neighbours.
- **Air links.** Airports appear with a probability that increases with wealth and density, and each receives a small random number of flights to other airports, drawn as quadratic Bézier arcs.

Finally the generator guarantees that the network is connected. A union–find structure with path compression [11] merges the endpoints of all links. While more than one component remains, the closest pair of coastal regions in different components is joined by a new sea link, creating ports where needed. No generated world can therefore be unwinnable by construction.

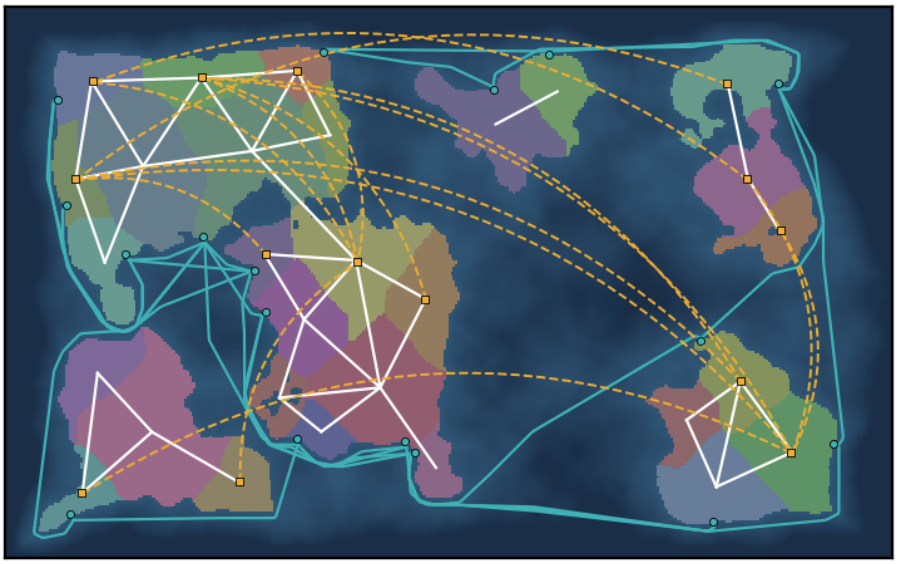


Figure 12: Transport network: land links (white), sea lanes (teal, ports as circles) and flights (orange dashed, airports as squares).

3.10 Maritime routing

Sea lanes must pass around landmasses and keep away from coasts. A truncated distance transform of the water is computed first, by a multi-source breadth-first search from all land cells over the 8-neighbourhood: $\varsigma(c)$ is the Chebyshev distance of water cell c to land, saturating at a small cap (Fig. 13a). Lanes are then found by A* [12] on the 8-connected water grid with step cost

$$c(p \to q) = \big(\sqrt{2} \text{ if diagonal, } 1 \text{ otherwise}\big) \cdot \omega(\varsigma_q), \qquad \omega \geq 1 \text{ decreasing with } \varsigma, \tag{10}$$

and the octile heuristic

$$\hat{h}(q) = \max(\Delta x, \Delta y) + (\sqrt{2} - 1)\min(\Delta x, \Delta y). \tag{11}$$

Since $\omega \geq 1$, $\hat{h}$ never exceeds the true remaining cost; it is admissible, so A* returns a minimum-cost lane [12]. The penalty ω pushes lanes offshore without forbidding narrow straits. The cell path is thinned and then smoothed by repeated passes of Chaikin's corner cutting [13], which replaces each segment P_iP_{i+1} by the points $\frac{3}{4}P_i + \frac{1}{4}P_{i+1}$ and $\frac{1}{4}P_i + \frac{3}{4}P_{i+1}$ and converges to the quadratic B-spline of the polygon (Fig. 14). Thanks to the heuristic, A* explores only a small fraction of the water cells (Fig. 13b).

(a) distance to shore ς (b) A* expansion order and lane



Figure 13: Sea-lane routing, cropped to the search area. (a) Distance-to-shore field ς (red: adjacent to land). (b) Cells expanded by A* in order (pink to yellow) and the smoothed lane (white).

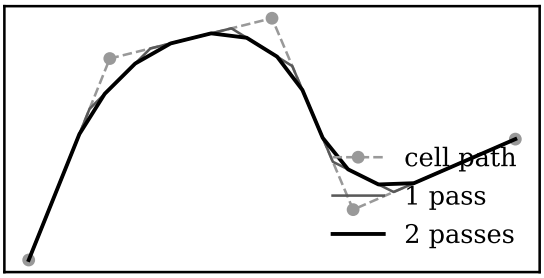


Figure 14: Chaikin corner cutting: one and two passes.

3.11 Determinism and incremental evaluation

The pipeline is written as a C# iterator that *yields* frequently inside its loops (per batch of rows, per batch of heap operations, per sea lane). Its caller resumes it until a fixed time budget is used up and then returns control to the engine (Section 7.2). Running the iterator to completion produces exactly the same world as a monolithic call; this was verified by hashing the region map and every sea lane for several seeds at both grid resolutions.

4 Epidemic simulation

The simulation is a discrete-time stochastic *metapopulation* model [14]: each region is a well-mixed population, and regions interact only through the transport links. One simulated day is advanced in the fixed order of Algorithm 3.

4.1 Time and pacing

All rates are multiplied by a global pace factor ρ, and every interval specified at unit pace is converted to simulated days by $\mathrm{Days}(d) = \mathrm{round}(d/\rho)$. This single factor sets the length of a session without changing the relative balance of any mechanic.

Algorithm 3: One simulated day

advance the environment: seasons, fronts, weather of each region (Algorithm 5);
foreach *reached region i that is not saturated* **do**
    grow I_i logistically (Eq. (12)); mark it overrun near saturation
foreach *link (a, b) and both directions* **do**
    with probability p_{ab} (Eq. (14)) launch a carrier $a \to b$
advance carriers; an arriving carrier infects its target and fixes its growth order (Algorithm 4);
if *research has not started* **then**
    raise suspicion A (Eq. (21)); start research when it saturates
else
    raise the countermeasure C (Eq. (22)); close links by paranoia; open laboratories
spawn collectible rewards and random events; in the endgame, seed remaining regions;
if *every region is overrun* **then** the player wins;
if *C is complete* **then** the player loses;

4.2 Growth inside a region

Let $I_i(t)$ be the infected population of region i and P_i its population. Growth follows the discrete logistic map [15]:

$$I_i(t+1) = I_i(t) + b_i(t)\, I_i(t)\Big(1 - \frac{I_i(t)}{P_i}\Big), \tag{12}$$

$$b_i(t) = \beta\, \rho\, g\, m_g\, r_{\text{clim}}\, r_{\text{wealth}}\, r_{\text{dense}}\, S_i(t)\; G_i(t)\; B_i(t), \tag{13}$$

where β is the base rate, g a difficulty factor, m_g the product of the player's growth upgrades, and the r factors express how hospitable each region type is (upgrades can raise them). The seasonal factor $S_i(t)$ and the weather factor $G_i(t)$ are defined by the environment model of Section 5 (Eqs. (29) and (33)): the first favours cold regions in their summer and hot regions in their winter, the second rewards the organism for weather it prefers. $B_i(t)$ collects temporary multipliers (special actions, events, the endgame of Section 4.7). A region is *overrun* when I_i comes within a small margin of P_i; it then saturates, rewards the player and destroys any laboratory in it. The trajectory has the familiar sigmoid shape: slow start, near-exponential middle, saturation (Fig. 15).

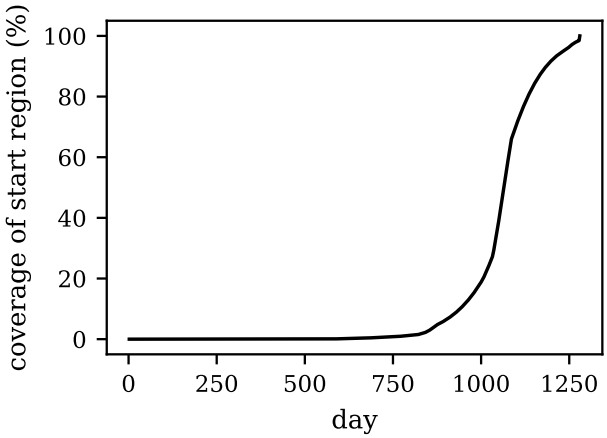


Figure 15: Coverage I/P of a starting region during a simulated session.

4.3 Transmission between regions

Every day each link is tried in both directions. A jump from a to b requires that the source has passed a minimum infected count, that the target is clean, and that no carrier is already heading to it. It then happens with probability

$$p_{ab} = \tau_{\text{type}}\, \rho\, \kappa\, f_a^{1/4}\, m_{\text{type}} \cdot \begin{cases} 1 & \text{link open} \\ \ell_{\text{type}} & \text{link closed} \end{cases}, \qquad f_a = I_a / P_a, \tag{14}$$

with a base rate τ per transport type, a difficulty factor κ, upgrade multipliers m, and a *leak* factor $\ell < 1$ for closed links. A link counts as open when the target keeps that transport open and the source either keeps it open or is already heavily infected. The sub-linear exponent makes even lightly infected regions leak noticeably, which keeps the early game moving. Weather scales p_{ab} by a further factor per transport type (Section 5.2). A success launches a *carrier* whose travel time is constant for land and proportional to route length (clamped to a range) for sea and air. On arrival it infects the target at an *entry cell*: the port, the airport, or for land the target cell nearest to the source. Figure 16 shows the spread over a session (Animation S2); the figure and the animation use a radial threshold field for clarity, the model of Section 4.4 changes the shape inside each region but not the sequence of regions.

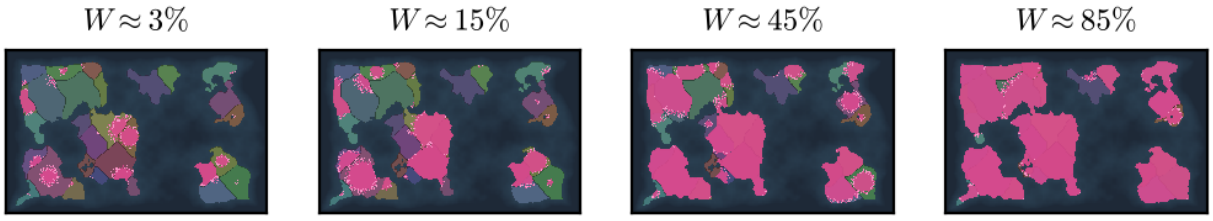


Figure 16: Spread over a simulated session at increasing world coverage W, drawn with a radial threshold field (Animation S2).

4.4 Sub-regional growth order

The rules track only one number per region, $f_i = I_i / P_i$. To show a colony that creeps over the land, every cell c of a region receives a fixed threshold $\vartheta_c \in [0, 1]$ at the moment the region is first reached; the region's visibility is a concave function of coverage,

$$v_i = v_0 + (1 - v_0)\, f_i^{1/3}, \qquad \text{and cell } c \text{ is drawn as colonised } \iff v_i > \vartheta_c. \tag{15}$$

Because v_i is non-decreasing in f_i and ϑ_c is fixed, the colonised set only grows; all the character of the spread is therefore in the field ϑ. A radial field (the normalised Euclidean distance to the entry cell plus white noise) would produce a disc with a speckled rim, the same in every region. The model instead derives ϑ from a *geodesic* arrival time on the terrain and then equalises it by rank.

Anisotropic cost. The region is treated as an eight-connected graph. Let $\tilde{h}_c \in [0, 1]$ be the height of cell c normalised within the region. Stepping from c to a neighbour c' in direction $\mathbf{d} = (d_x, d_y)$, $d_x, d_y \in \{-1, 0, 1\}$, costs

$$\begin{aligned} w(c \to c') = \|\mathbf{d}\| \; & \underbrace{\left(1 + h_r \left(k_1 \tilde{h}_{c'} + k_2 \left[\tilde{h}_{c'} - \tilde{h}_c\right]_+\right)\right)}_{\text{relief}} \; \underbrace{\max\left(c_{\min},\, 1 + h_p \left(n_p(c') - \bar{n}\right)\right)}_{\text{poor and rich ground}} \\ & \times \underbrace{\left(1 - h_v\, \zeta(c')\right)\left(1 + h'_v \left(1 - \zeta(c')\right)\right)}_{\text{veins}} \; \underbrace{\left(1 - h_w\, d_x\right)}_{\text{wind}}, \end{aligned} \tag{16}$$

where $[\cdot]_+$ is the positive part (climbing is penalised, descending is not), $n_p \in [0,1]$ is a two-octave value noise (Eq. (3)) with mean level $\bar{n}$, the floor $c_{\min} > 0$ keeps the cost positive, $h_v < 1$ and $h_w < 1$ for the same reason, and

$$\zeta(c) = \max\bigl(0,\, 1 - k_\zeta \,|2\, n_v(c) - 1|\bigr) \tag{17}$$

is close to one on the level set $n_v = \frac{1}{2}$ of a second, smoother noise. Those level sets form a network of thin curves, so the term makes a web of cheap "veins" along which the front runs ahead as tendrils; off the veins the cost is raised slightly (h_v') so that the mean speed is preserved. The wind term makes steps towards the east ($d_x = 1$) cheaper and steps towards the west dearer. The vector (h_r, h_p, h_v, h_w) together with the number of secondary seeds is the *habit* of an organism variant: an even crust (small h_p and h_v, almost no relief penalty), a flowing net (large h_v and h_r), a wind-borne variant ($h_w > 0$), a spore-bursting variant (many seeds), and so on.

Secondary seeds. A first Dijkstra pass [9] from the entry cell e_i yields arrival times $T_0(c)$. Then up to n_s further cells are chosen with T_0 in a middle band of its range and at a minimum mutual distance, and each is given the start time $\varrho\, T_0(c)$ with $\varrho < 1$. A second pass with all seeds gives the final arrival time

$$T(c) = \min_{(s,t_s)\in\mathcal{S}} \bigl(t_s + \mathrm{dist}_w(s,c)\bigr). \tag{18}$$

Since a secondary seed starts before the front from e_i would reach it, an isolated colony appears ahead of the front and later merges with it (Algorithm 4). This is the discrete analogue of a first-arrival (eikonal) problem with several sources [16].

Algorithm 4: Growth order of a newly reached region

Input: cells of the region, entry cell e, habit (h_r, h_p, h_v, h_w), seed count n_s, factor ϱ

$T_0 \leftarrow$ DIJKSTRA$(\{(e,0)\})$ with step cost Eq. (16);
$\mathcal{S} \leftarrow \{(e,0)\}$;
while $|\mathcal{S}| < 1 + n_s$ *and the attempt limit is not reached* **do**
  pick a cell c by a hash of the region index and the attempt number;
  if $T_0(c) / \max T_0 \in [\tau_1, \tau_2]$ *and* c *is far from every seed in* $\mathcal{S}$ **then** $\mathcal{S} \leftarrow \mathcal{S} \cup \{(c, \varrho\, T_0(c))\}$;
$T \leftarrow$ DIJKSTRA$(\mathcal{S})$;
sort the cells by T (ties broken by a fixed per-cell hash); let $r_c \in [0,1]$ be the rank of c divided by the number of cells minus one;
$\vartheta_c \leftarrow v_0 + (1 - v_0)\, r_c^{2/3}$;

Rank equalisation. Arrival times have a long upper tail: one distant or steep corner can stretch the scale so that most cells have a small normalised time, and the region would look taken long before it is. Replacing T by its rank removes the tail and makes the displayed area a known function of the true coverage. With

$$\vartheta_c = v_0 + (1 - v_0)\, r_c^{2/3} \tag{19}$$

and Eq. (15), cell c is drawn when $f^{1/3} > r_c^{2/3}$, i.e. when $r_c < \sqrt{f}$. As the ranks are uniformly spaced in $[0,1]$, the share of the region drawn as colonised is

$$\mathcal{A}(f) = \sqrt{f} \tag{20}$$

for every region, every habit and every terrain (Fig. 18). The square root keeps small infections visible while guaranteeing that a region is completely covered exactly when it is completely held. The cost is two heap-based Dijkstra passes and one sort, $O(n \log n)$ for a region of n cells, paid once when the region is reached. The comparison of Eq. (15) is evaluated per pixel on the GPU (Section 6.2); the CPU only supplies v_i.

Coupling with the rules. The construction draws nothing from the random stream of the rules: its only random inputs are hashes of the world seed. The field ϑ is nevertheless read by one rule, the placement of collectible rewards, which samples cells of a region by rejection until it finds a colonised one and therefore consumes a number of draws that depends on ϑ. Two growth models thus give sessions that are equal in distribution but not draw for draw; Section 8.2 verifies that the measured balance is the same within noise.

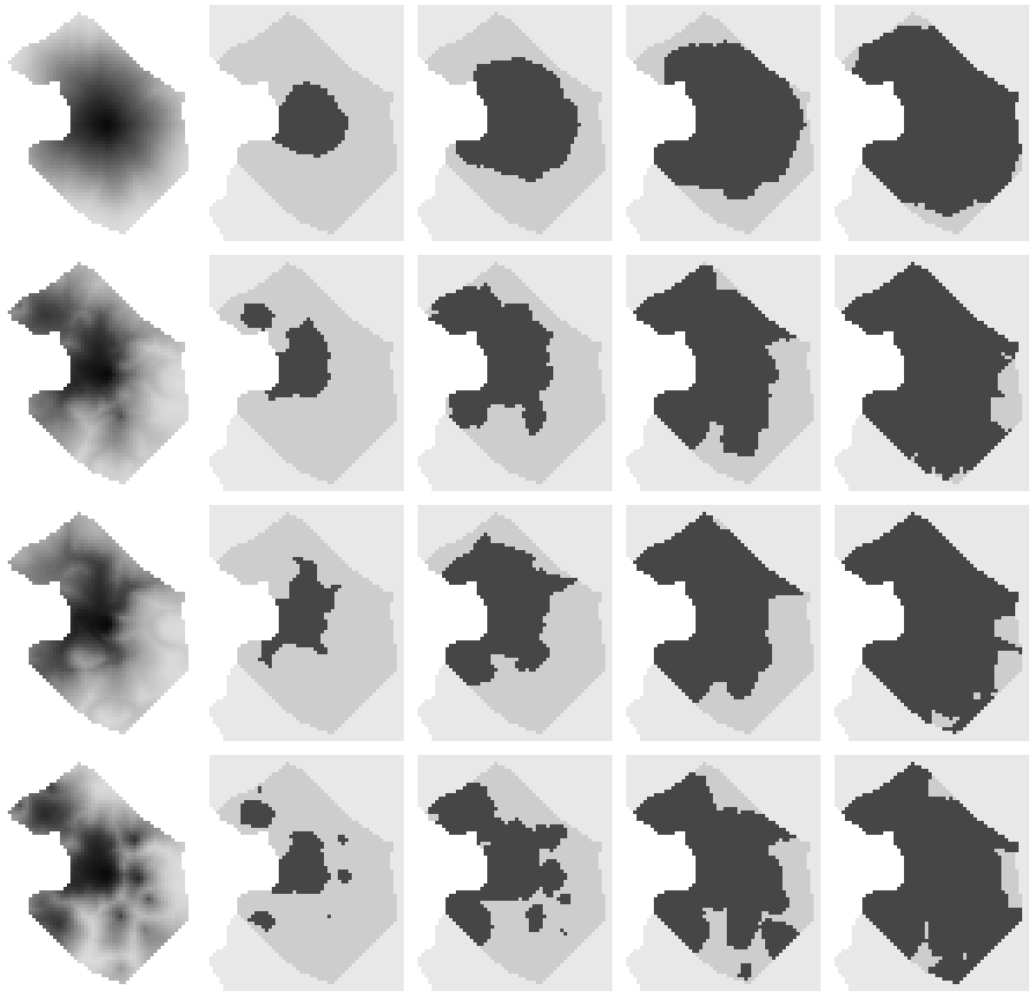


Figure 17: Growth order for four habits in the same region (rows: even crust, default, strongly veined, spore-bursting). First column: the threshold field ϑ (dark: early). Other columns: the colonised set at increasing coverage f.

4.5 Detection and countermeasure

Before research starts, a suspicion level A grows each day with the world coverage $W = \sum_i I_i / \sum_i P_i$:

$$A \leftarrow A + \rho\, \alpha\, m_{\mathrm{vis}}\, (1 + \chi_A\, \nu)\, \left(a_0 + a_1 \sqrt{W}\right), \tag{21}$$

where ν is the colony's *noise*, a trade-off chosen by the player through upgrades (Section 4.8). Research begins when A saturates or when W passes a notice level. From then on the countermeasure C advances as

$$C \leftarrow C + \rho\, c_0\, c\, (1 + \chi_C\, \nu)\; E\; \left(h_0 + (1 - h_0)\, \mathcal{H}\right)\; m_C\, (1 + \lambda_{\mathrm{lab}}\, n_{\mathrm{lab}}), \tag{22}$$

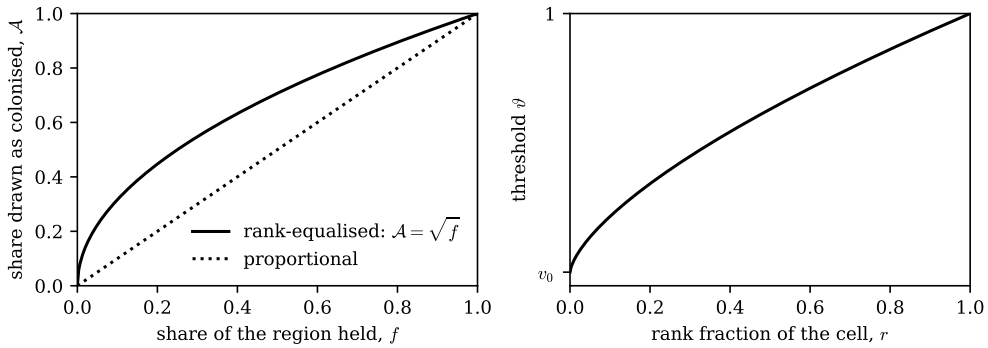


Figure 18: Left: the share drawn as colonised as a function of the share held, Eq. (20). Right: the threshold assigned to a cell as a function of its rank, Eq. (19).

where n_{lab} is the number of open laboratories, λ_{lab} their strength, and $\mathcal{H}$ the wealth-weighted share of the world that is still healthy:

$$\mathcal{H} = \frac{\sum_i w_i (1 - f_i)}{\sum_i w_i}, \qquad w_i \text{ increasing with wealth.} \tag{23}$$

The factor E is an adaptive *rubber band*. With the lead $\Delta = C - 100\,W$ (in percentage points) and n_b the number of upgrades bought,

$$E = \begin{cases} \max\!\big(1 - a \min(1, n_b / n^*),\; 1 - \delta(\Delta - \Delta_+)\big) & \Delta > \Delta_+ \\ 1 + \chi \min\!\big(1, (-\Delta - \Delta_-)/\Delta_s\big) & \Delta < -\Delta_- \\ 1 & \text{otherwise.} \end{cases} \tag{24}$$

Research slows down when it is far ahead and speeds up when the player is far ahead, which keeps sessions competitive across skill levels without erasing the reward for good play. Figure 19 shows W, A and C over a simulated session.

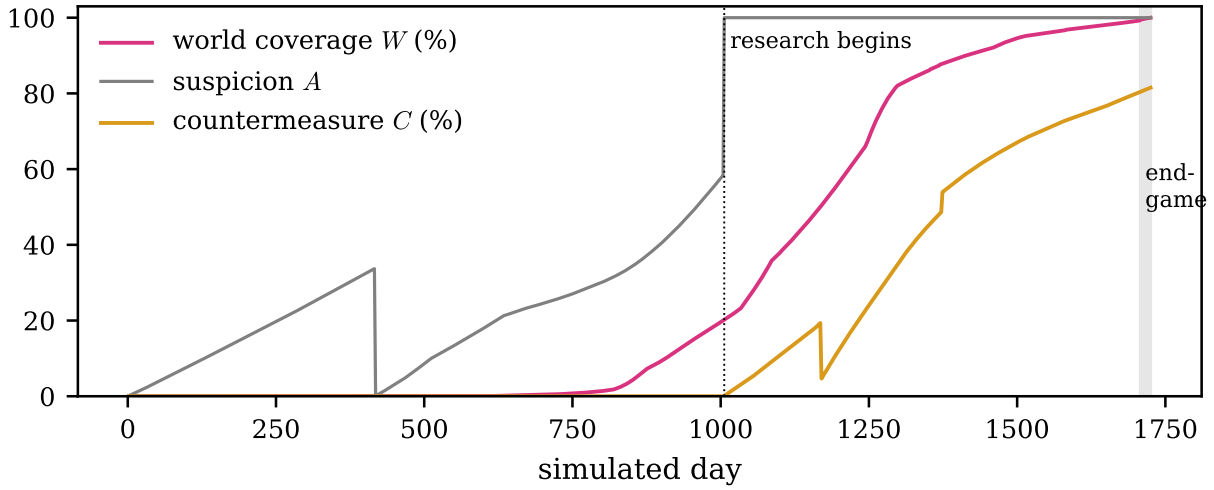


Figure 19: World coverage W, suspicion A and countermeasure C over a simulated session. The dotted line marks the start of research; the shaded band is the endgame.

4.6 Containment: closures and laboratories

Once research runs, every region compares world coverage with its paranoia π_i, scaled by difficulty and by the player's upgrades. It closes its airport, then its port, then its land borders

at increasing multiples of π_i, one closure per region per day (Fig. 20). Above a collapse level of world coverage, containment breaks down: no further closures happen and closed links stop holding. Laboratories open at random intervals up to a maximum number. Their host is drawn by roulette-wheel selection with weights that favour rich, healthy regions, and each open laboratory accelerates the countermeasure. The player can wreck a laboratory in a reached region, and overrunning its region destroys it.



Figure 20: Containment during research: closed airports (orange ×), closed ports (teal ×), closed land borders (red rings) and laboratories (white triangles).

4.7 Endgame

When only a small number of regions (a fixed fraction of N_R, with a minimum) remain unsaturated, an *endgame* phase begins: growth is multiplied and, at regular intervals, a random unreached region is infected directly. This removes the long tail in which the last isolated regions would take a very long time to fall, while still requiring the player to have won the race against the countermeasure.

4.8 Player economy

Collectibles. The player's currency comes from collectibles that appear in reached regions at random intervals that shorten as the colony spreads. They must be collected within a short time; a missed one pays only a fraction. Reaching and overrunning regions also pay.

Upgrades. Upgrades are arranged in several hexagonal trees with prerequisites. The price of upgrade n is

$$\text{price}_n = \text{round}\big((c_n + q\,O)\,\kappa_{\text{cost}}\big) + s_n\,o_n, \tag{25}$$

where c_n is its base price, O the total number of upgrades owned (so that every purchase makes the next one dearer), q the per-purchase increment, and s_n an optional step applied per copy o_n already owned. Upgrades act multiplicatively on the factors of Eqs. (13), (14), (21) and (22). Selling one refunds part of its price. The noise ν is a deliberate trade-off: noisy upgrades make collectibles more frequent but also raise suspicion and research speed.

Mutations and variants. When the number of reached regions passes successive milestones, the player drafts one of three random mutations; redrawing costs an increasing price. Several

organism variants, unlocked by progress, change passive factors and provide a special action with a cost, a cooldown and a duration.

4.9 Outcomes

The player wins when every region is overrun and loses when the countermeasure completes. Apart from upgrade effects and random events, both W and C are monotone, so a session is a race between the logistic spread and the research process, modulated by containment and the rubber band.

5 Environment model: seasons, weather and ground memory

The environment is a second, slower dynamical system that runs beside the epidemic. It has three layers: a deterministic *seasonal clock*, a stochastic field of moving *weather fronts*, and a per-location *ground state* that integrates the weather over time. The first two enter the growth, transmission and detection rates of Section 4; the third is purely visual and is what makes the land appear to remember the weather (Section 6.3).

5.1 Seasonal clock and hemispheres

Let Y be the length of a year in simulated days and

$$\psi(t) = 4\,\frac{t \bmod Y}{Y} \in [0, 4) \tag{26}$$

the *season phase* of the northern half of the map; its integer part is the season index $k \in \{0, 1, 2, 3\}$ (spring, summer, autumn, winter). With the row coordinate y counted from the northern edge, the southern half is half a year out of phase,

$$\psi_h(t) = \big(\psi(t) + 2\,[h = \text{south}]\big) \bmod 4, \qquad h(y) = \begin{cases} \text{north} & y < H/2 \\ \text{south} & \text{otherwise,} \end{cases} \tag{27}$$

so that the two polar belts are never in winter at the same time. A region belongs to the hemisphere of its centroid. A continuous temperature term is centred on the middle of summer,

$$\sigma_h(t) = \pm \sin\!\Big(2\pi\,\frac{t - Y/8}{Y}\Big), \tag{28}$$

with the positive sign in the north. The shift by $Y/8$ (half a season) puts the maximum in mid-summer and the zero crossings in mid-spring and mid-autumn; an unshifted sine would peak on the first day of summer and contradict the season index (Fig. 21, right). The seasonal growth factor of Eq. (13) is

$$S_i(t) = \big(1 + \varepsilon_i\,\sigma\,\sigma_{h(i)}(t)\big)\;\mathsf{a}_{k(i,t)}, \qquad k(i,t) = \lfloor \psi_{h(i)}(t) \rfloor, \tag{29}$$

where $\sigma \in (0, 1)$ is the seasonal amplitude, $\varepsilon_i \in \{+1,\ 0,\ -\varepsilon_h\}$ for cold, temperate and hot regions (cold regions are most hospitable in their summer, hot regions in their winter, with a smaller amplitude $\varepsilon_h < 1$), and $\mathsf{a}_k > 0$ is the *season affinity* of the organism variant in season k, a vector of four multipliers that the rules may change during play. Rendering needs smooth transitions, so it does not use the integer season but four overlapping triangular basis functions on the circle,

$$\Omega_k(\psi) = \max\!\Big(0,\ 1 - \big|\,4\,\mathrm{frac}\big(\tfrac{\psi - c_k}{4} + \tfrac{1}{2}\big) - 2\,\big|\Big), \qquad c_k = k + \tfrac{1}{2}, \tag{30}$$

each equal to one in the middle of its season and to zero in the middle of the neighbouring ones; they form a partition of unity, $\sum_k \Omega_k = 1$ (Fig. 21, left). The southern weights are the same functions with indices rotated by two, and in a narrow band around the equator the two sets are blended with a smooth step in y, so that no seam is visible.

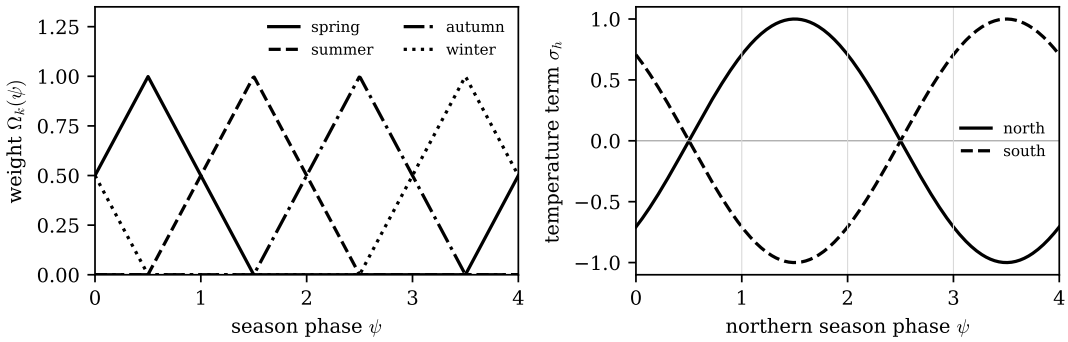


Figure 21: Left: the four season weights of Eq. (30). Right: the temperature term of Eq. (28) in the two hemispheres.

5.2 Weather fronts as a marked point process

Weather is carried by a small set $\mathcal{F}$ of *fronts*. A front $F = (\mathbf{x}, \mathbf{u}, R, \varkappa, p, t_{\text{end}})$ is a disc of radius R centred at $\mathbf{x}$ that drifts with velocity $\mathbf{u}$ (predominantly from west to east, with a small meridional component), has a kind $\varkappa \in \{\text{rain}, \text{storm}, \text{snow}, \text{heat}, \text{fog}\}$, a strength $p \in (0, 1]$ and an end time. Each day every front is advected, $\mathbf{x} \leftarrow \mathbf{x} + \mathbf{u}$, and removed when it expires or leaves the map. While fewer than n_F fronts exist, a new one is created after a random waiting time; it enters at the western edge with probability q_w and at a random position otherwise.

The kind of a new front is drawn from a categorical distribution whose weights depend on the latitude ℓ of its spawn point (Section 3.8) and on the season *k of its own hemisphere* (Eq. (27)):

$$\Pr[\varkappa = j \mid \ell, k] = \frac{\xi_j \, c_j(\ell, k)}{\sum_{j'} \xi_{j'} \, c_{j'}(\ell, k)}, \qquad \ell = 2 \left| \, y/H - \tfrac{1}{2} \right|. \tag{31}$$

The table $c_j(\ell, k) \geq 0$ encodes the climatology: rain peaks in the transitional seasons and is rare over the polar belt outside its summer; snow dominates the polar belt in winter; heat belongs to the equatorial belt and to temperate summers; fog to the cool seasons outside the tropics. The factor $\xi_j \geq 1$ is a per-kind bias that the rules can raise for a given organism variant. The fronts draw from a pseudo-random stream of their own (Section 3.1, with a different seed), not from the stream of the epidemic, so the weather model can be changed without altering the sequence of draws of the rules.

Weather seen by a region. Region i with centroid $\mathbf{c}_i$ is under front F when $d = \|\mathbf{c}_i - \mathbf{x}_F\| / R_F < 1$, with amount $a = p_F \, (1 - d^2) \in (0, 1]$. The region takes the kind $\varkappa_i$ and amount a_i of the front with the largest amount; with no front, $a_i = 0$. Three corrections are then applied, both in the rules and, per texel, in the rendered weather field:

$$\varkappa_i \leftarrow \begin{cases} \text{snow} & \varkappa_i = \text{rain}, \ \text{climate cold}, \ \text{local season} \neq \text{summer} \\ \text{rain} & \varkappa_i = \text{snow}, \ \text{climate hot} \\ \text{none} \ (a_i \leftarrow 0) & \varkappa_i = \text{heat}, \ \text{climate cold}. \end{cases} \tag{32}$$

Algorithm 5: One day of the environment

foreach *front* $F \in \mathcal{F}$ **do**
  $\mathbf{x}_F \leftarrow \mathbf{x}_F + \mathbf{u}_F$; remove F if it has expired or left the map
if $|\mathcal{F}| < n_F$ *and the waiting time has elapsed* **then**
  draw a spawn row y; $k \leftarrow \lfloor \psi_{h(y)} \rfloor$; draw $\varkappa$ from Eq. (31);
  draw radius, strength, drift and lifetime; place the front at the western edge with probability q_w;
  draw the next waiting time;
foreach *region* i **do**
  $(\varkappa_i, a_i) \leftarrow$ kind and amount of the strongest front covering the centroid $\mathbf{c}_i$ (none: $a_i = 0$);
  apply the corrections of Eq. (32);
foreach *texel of the weather lattice* **do**
  rasterise the fronts with the same corrections; update the ground state by Eqs. (34) to (36)

Thus precipitation over cold land falls as snow except in the local summer, snow never reaches the hot belt, and heat waves do not reach the ice. Since the generator never places a hot region next to a cold one (Section 3.8), the corrections cannot produce a snowfield bordering a heat wave.

Effect on the epidemic. With amount a_i and kind $\varkappa_i$ the growth rate of Eq. (13) is multiplied by

$$G_i = 1 + a_i \left(G^0_{\varkappa_i} \, \Theta_{\varkappa_i} \, m_{\varkappa_i} - 1 \right), \tag{33}$$

where $G^0_{\varkappa} > 0$ is the base effect of the kind (above one if the kind is favourable, below one if it is adverse), $\Theta_{\varkappa}$ the taste of the organism variant for it and $m_{\varkappa}$ a further multiplier acquired during play; $G_i = 1$ in clear weather, and the factor interpolates linearly with the amount. The transmission probability of Eq. (14) is scaled in the same way by a factor that depends on the kind *and* on the transport type, (storms hinder flights, for example). Fog acts on detection instead: the daily increment of suspicion in Eq. (21) is reduced in proportion to the share of the infected population that is currently under fog.

5.3 Ground state: the memory of the weather

A coarse lattice (one texel per block of cells) carries three state variables per texel, wetness w, snow depth s and dryness d, all in $[0, 1]$. They are advanced once per simulated day by first-order relaxation equations driven by the local rain r, snowfall n and heat e (each in $[0, 1]$) of the corrected weather field:

$$\mathsf{w}' = \operatorname{clamp}\left(r_w \, \mathsf{w} + \alpha_r \, r - \alpha_e \, e \right), \tag{34}$$

$$\mathsf{s}' = \operatorname{clamp}\left(\mathsf{s} + \alpha_n \, n - \max(\mu_0, \, \mu) \right), \tag{35}$$

$$\mu = \mu_1 + \mu_s \, \Omega_{\text{summer}} + \mu_e \, e + \mu_r \, r + \mu_c \, (1 - \chi) - \mu_w \, \chi \, \Omega_{\text{winter}},$$

$$\mathsf{d}' = \operatorname{clamp}\left(r_d \, \mathsf{d} + \alpha_h \, e - \alpha_d \, r \right), \tag{36}$$

where $r_w, r_d \in (0, 1)$ are retention factors, the α and μ are positive gains, $\chi \in \{0, \frac{1}{2}, 1\}$ is the coldness of the climate at the texel (hot, temperate, cold), the Ω are the season weights of the texel's hemisphere, and clamp limits to $[0, 1]$. Wetness and dryness are leaky integrators: without

forcing they decay geometrically with time constants $1/(1-r_w)$ and $1/(1-r_d)$ days. Snow is a reservoir with a melt rate μ that grows in summer, in warm climates, under rain and under heat, and falls to the floor μ_0 in a cold winter, so that snow persists there between snowfalls and disappears within days elsewhere. The lattice is uploaded as a three-channel texture and sampled with bilinear filtering, so the state costs one small texture update per simulated day.

6 Rendering

The picture is built from a small number of full-map textures written once per world, a few small textures that change during play, and three shaders (land, water, vegetation). The guiding rule is that everything on screen is drawn with *flat tones and crisp, slightly irregular edges*; soft gradients are reserved for effects that span the whole scene (night, depth). Procedural shapes are therefore evaluated analytically and anti-aliased with screen-space derivatives: for a scalar field g and a level τ,

$$\mathrm{step}_{\mathrm{aa}}(g,\tau) = \mathrm{smoothstep}\big(\tau-\epsilon,\ \tau+\epsilon,\ g\big), \qquad \epsilon = k_\epsilon\big(|\partial_x g| + |\partial_y g|\big), \tag{37}$$

where the derivatives are taken with respect to screen pixels, so that an edge is about one pixel wide at any zoom [19].

6.1 Base map

At world creation the CPU paints a base texture of the land at a fixed number of pixels per cell, reduced on low-end devices. It contains biomes, terraced relief, rivers, borders and small decorations. Painting proceeds in row batches inside the same time-sliced iterator as generation (Section 3.11).

Biomes. A pixel's biome is a function of the climate of its region, its height $\tilde{h}$ above sea level (normalised to $[0,1]$) and a moisture noise $\mathcal{M}$: thresholds on $(\tilde{h}, \mathcal{M})$ select among open, wooded, wet, arid and rocky classes for each climate. Snow caps exist only above a height threshold *and* outside hot climates: a hot summit is bare rock, and temperate mountains have a band of bare rock below the snow line. Each biome has a flat base colour and an analytic texture of its own, thresholded rather than blended.

Relief. Height is quantised into a few terraces; each step is a touch lighter than the one below. On steep steps the painter adds *hachures*: with the height gradient ∇h and the unit tangent to the contour $\mathbf{t} = \nabla h^{\perp}/\|\nabla h\|$, a pixel at position $\mathbf{x}$ on a terrace edge is darkened when

$$\mathrm{frac}\big(k_h\,\mathbf{x}\cdot\mathbf{t}\big) < \tau_h \quad \text{and} \quad \|\nabla h\| > g_{\min}, \tag{38}$$

that is, at regular intervals *along* the contour; since the marks are confined to the narrow edge band, they appear as short strokes pointing down the slope, as on a hand-drawn map.

Smooth borders from a cell partition. Regions are sets of cells, so drawing their borders along cell edges gives staircases. Instead each pixel takes part in a weighted vote among the nine nearest cells with a separable quadratic B-spline kernel [17],

$$B(x) = \begin{cases} \frac{3}{4} - x^2 & |x| < \frac{1}{2} \\ \frac{1}{2}\big(\frac{3}{2} - |x|\big)^2 & \frac{1}{2} \le |x| < \frac{3}{2} \\ 0 & \text{otherwise,} \end{cases} \qquad V_r(\mathbf{x}) = \sum_{c:\ \mathrm{reg}(c)=r} B(x-x_c)\,B(y-y_c), \tag{39}$$

with $\mathbf{x} = (x, y)$ in cell units. The kernel has support three cells wide, which is why nine cells suffice, and its integer translates sum to one.

Algorithm 6: Region of a pixel by weighted vote

Input: pixel position $\mathbf{x}$ in cell units, cell partition $\mathrm{reg}(\cdot)$

$\mathbf{c}_0 \leftarrow \mathrm{round}(\mathbf{x})$; clear the list of candidates (at most nine);

for *each of the* 3×3 *cells* c *around* $\mathbf{c}_0$ *that is land* **do**

  $k \leftarrow B(x - x_c)\, B(y - y_c)$; add k to the entry of $\mathrm{reg}(c)$ in the list (linear search);

$r^* \leftarrow$ entry with the largest sum $V_{(1)}$; $V_{(2)} \leftarrow$ second largest sum (zero if there is none);

return r^* *and the margin* $\Delta_V = (V_{(1)} - V_{(2)}) / \sum_r V_r$

The pixel belongs to $r^* = \arg\max_r V_r$, and the normalised margin $\Delta_V \in [0, 1]$ between the two best regions measures how close the vote was. Fills (biome, tint) use r^*; the border line is drawn where $\Delta_V < \tau_b$, with a soft edge. Because every V_r is C^1, the set $\{\Delta_V = \tau_b\}$ is a smooth curve that approximates the partition instead of following its cell edges (Fig. 22). The outlines drawn at run time (selected, hovered and closed regions) use the same idea in the shader: the $0/1$ flag of the four nearest cells is interpolated bilinearly and the outline is the band where the interpolated value crosses one half, so painted and dynamic lines coincide.

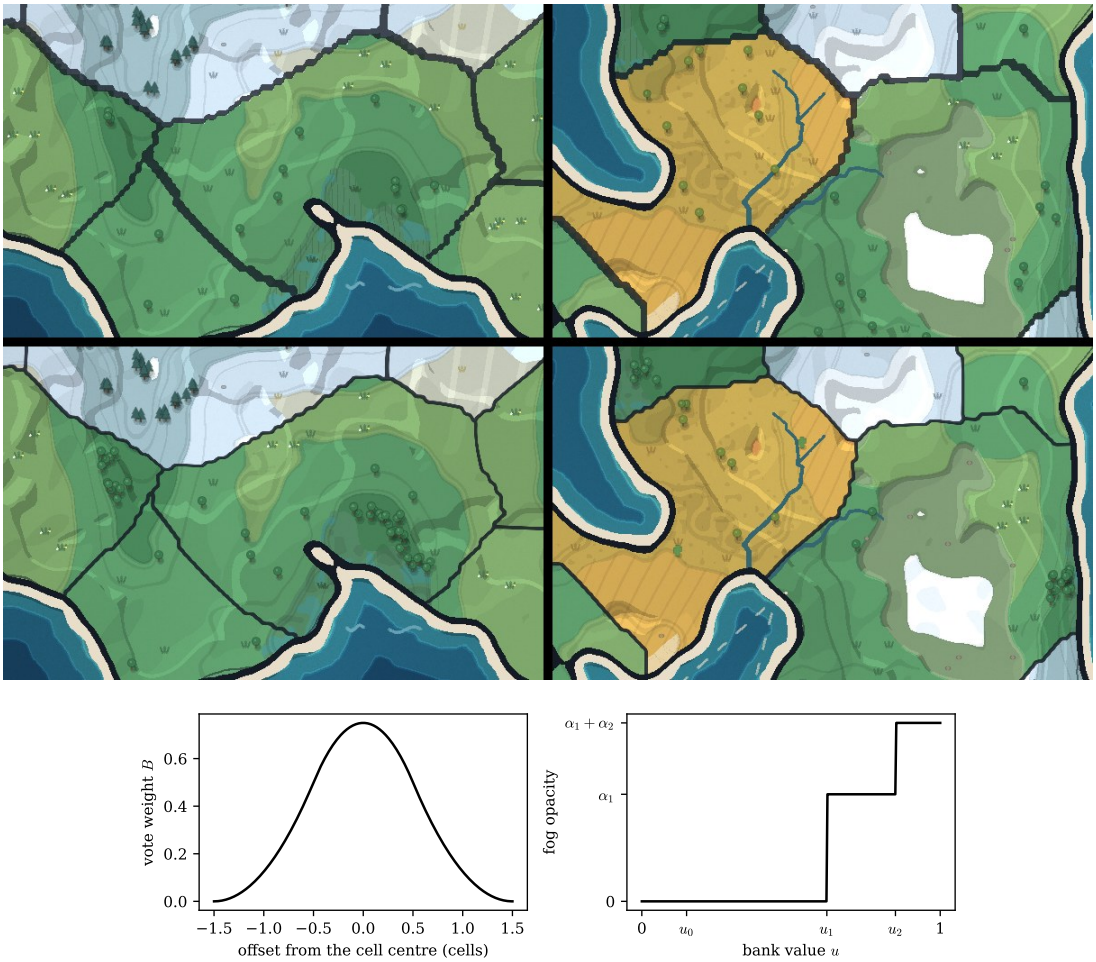


Figure 22: Top: borders along cell edges (upper row) and from the vote of Eq. (39) (lower row). Bottom left: the kernel B. Bottom right: the two-level fog transfer function of Eq. (47).

6.2 GPU colony shader

The colony is not painted by the CPU. Three data textures encode everything the shader needs:

- a *cell* texture with the threshold ϑ_c (16-bit) and the region index of each cell; sea cells within a fixed number of rings of the coast inherit the threshold of the nearest land cell, with the ring number in a spare channel, so that the colony can spread onto the submerged shelf (Section 6.4) with a delay per ring;
- a *static* texture with the land mask, grain and a per-cell hash;
- a *region* texture, a single row with the visibility v_i, state flags (overrun, closed, selected, hovered) and a light level (Section 6.5) of every region.

Each frame the CPU uploads only the region row. The fragment shader interpolates the signed margin $d = v_i - \vartheta_c$ bilinearly between the four nearest cells, adds a small grain, and draws the colony where $d > 0$ with opacity $\mathrm{sat}(k_d\, d)$; the narrow band $0 < d < d_f$ is drawn darker as the *front*. The cell texture changes only when a region is first reached (Section 7.4).

Stain and motifs. Behind the front the colony is a translucent stain that lets the ground show through: its colour is multiplied by a clamped affine function of the luminance L of the land below,

$$\mathbf{c} = \mathbf{c}_{\text{variant}} \cdot \mathrm{lerp}\big(1,\ \mathrm{clamp}(\lambda_0 + \lambda_1 L,\ \lambda_{\min}, \lambda_{\max}),\ \lambda_m\big), \qquad \lambda_{\min} < 1 < \lambda_{\max}, \tag{40}$$

so terraces and biomes remain legible, while the clamp prevents the colony from turning pale on snow or muddy in shade. Small drawings (*motifs*) of the variant are then scattered over it from an atlas. The plane is divided into square patches of side s_p cells; patch $\mathbf{j} \in \mathbb{Z}^2$ carries a motif with probability p_m, decided by a hash of $\mathbf{j}$, at a hashed centre $\mathbf{o}_\mathbf{j}$, half-size $\varsigma_\mathbf{j}$ and rotation $\alpha_\mathbf{j}$, with ranges chosen so that the rotated square stays inside the patch; no motif is therefore cut by a patch edge. The local coordinate

$$\mathbf{p} = \mathbf{R}(\alpha_\mathbf{j})\, \frac{\mathrm{frac}(\mathbf{x}/s_p) - \mathbf{o}_\mathbf{j}}{\varsigma_\mathbf{j}} \in [-1,1]^2 \tag{41}$$

indexes one atlas cell, and pixels with $\mathbf{p} \notin [-1,1]^2$ show no motif. Since $\mathbf{p}$ is discontinuous at patch edges, implicit derivatives would select a wrong mip level there; the atlas is therefore sampled with explicit gradients, the derivatives of the continuous coordinate $\mathbf{x}$ scaled by the constant factor between $\mathbf{x}$ and the atlas coordinate [18]. Two such layers, offset by half a patch, avoid a visible lattice.

6.3 Seasons and ground state on the land

A *class* texture gives, per cell, how much of the ground is deciduous wood (c_w), grass (c_g) or arid (c_a), each in $[0,1]$, and the coldness χ of the region. With the hemisphere-blended season weights Ω_k of Eq. (30), the land colour $\mathbf{L}$ is transformed by four successive convex blends, one per season, each towards a flat target tone scaled by the luminance L of the pixel:

$$\mathbf{L} \leftarrow \mathrm{lerp}\big(\mathbf{L},\ \mathbf{T}_k(L, \Pi),\ \Omega_k\, K_k(c_w, c_g, c_a)\big), \qquad k = 0, \ldots, 3, \tag{42}$$

where $K_k \in [0,1]$ states how strongly season k acts on each class (for example autumn on deciduous wood, summer on grass) and Π is a low-frequency *patch* noise, so that groves turn and grass ripens patch by patch. At most two of the Ω_k are non-zero at any time, so in practice two blends are active. Small crisp dots (one hashed point per tile, drawn with Eq. (37)) add

flowers in spring and fallen leaves in autumn. The ground state of Section 5.3 then acts: dryness d blends grass towards straw; wetness w raises saturation, lowers value and opens puddles where a fine noise exceeds $1 - k_w \mathrm{w}$; and snow covers the pixel where

$$\Pi_s(\mathbf{x}) < \Theta_s, \qquad \Theta_s = \mathrm{sat}\big(\max(\Omega_{\text{winter}}\, \Psi(\chi),\, k_s\, \mathrm{s})\big), \tag{43}$$

with $\Pi_s \in [0,1]$ a fixed noise and Ψ an increasing function of the coldness with $\Psi(0) = 0$. The first argument is the seasonal snow of cold climates, the second the snow that actually fell. Because the cover is a threshold on a fixed noise, snow appears as patches that grow and merge as Θ_s rises and retreat along the same contours as it falls (Fig. 23). Billboard vegetation reads the same textures at its base point, per vertex, to take the leaf tint of its hemisphere and to carry snow.

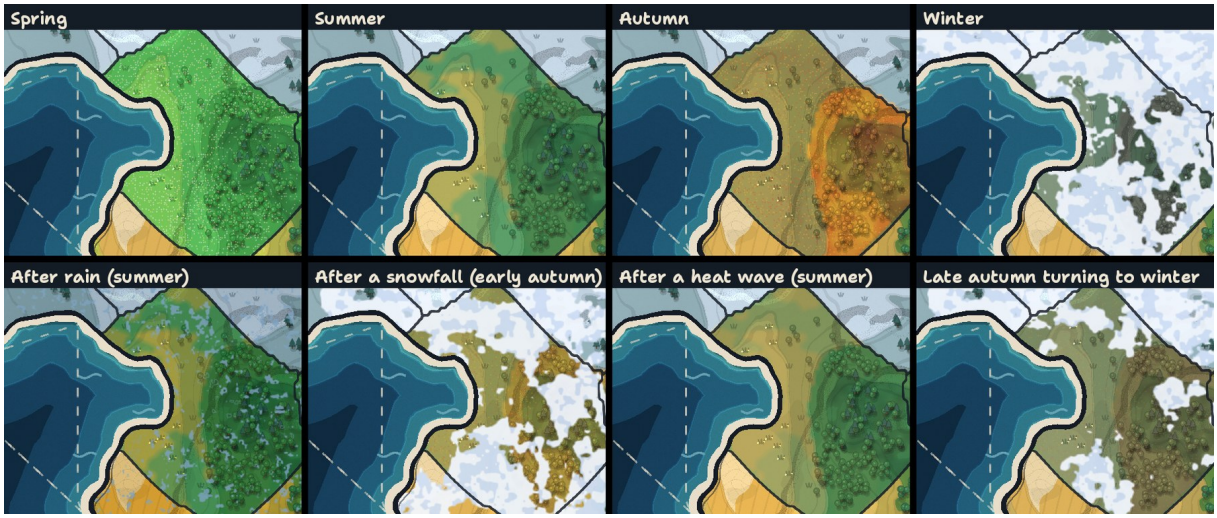


Figure 23: One location through the year and after three kinds of weather, computed with Eqs. (42) and (43) on the painted base map.

6.4 Water

The sea is a separate plane at sea level. A distance transform of the water, as in Section 3.10 but with a larger cap, gives for each sea cell the distance D to the nearest land, stored as a texture; the flood fill from the map edge of Section 3.9 separates the ocean from lakes.

Shelf and tones. The land continues under water as a shelf of irregular width $\Sigma(\mathbf{x}) = \Sigma_0 + \Sigma_1 n_1(\mathbf{x}) + \Sigma_2 n_2(\mathbf{x})$, with two noises n_1, n_2 of different frequency. Water colour is a stack of flat tones $\mathbf{t}$ selected by the anti-aliased thresholds of Eq. (37) on a slightly perturbed distance $\tilde{D}$, and a matching *cover* $\Gamma \in [0,1]$ states how much of the bottom the water hides (little over the shelf, everything in the deep):

$$\mathbf{c} = \mathrm{lerp}\big(\mathbf{b}(\mathbf{x}_b),\, \mathbf{t}(\tilde{D}),\, \Gamma(\tilde{D})\big), \tag{44}$$

where $\mathbf{b}(\mathbf{x}_b)$ is the colour of the bottom at the point seen through the surface (below). The deepest tone equals the background colour of the frame, so the sea has no visible end at the edge of the map.

Parallax and refraction without a scene copy. The bottom is not read back from the frame buffer; it is evaluated analytically at the point where the view ray meets it. With the unit vector

$\mathbf{v} = (v_x, v_y, v_z)$ from the surface point to the camera (y up) and a bottom depth $z_b(\mathbf{x}) \geq 0$ (small on the shelf, growing beyond it),

$$\mathbf{x}_b = \mathbf{x} - \frac{(v_x,\ v_z)}{\max(v_y,\ v_{\min})}\, n_r\, z_b(\mathbf{x}) \ +\ k_r\ \min(1, z_b)\ \big(\nabla\eta(\mathbf{x}, t) + \mathbf{r}(\mathbf{x}, t)\big). \tag{45}$$

The first term is the horizontal travel of the ray between the surface and the bottom: a ray leaving the surface downwards along $-\mathbf{v}$ moves horizontally by $(v_x, v_z)/v_y$ per unit of depth; the factor $n_r < 1$ shortens it, as refraction towards the normal would, and $v_{\min}$ bounds it at grazing angles. The second term is a small offset along the slope of an animated swell η plus a fine ripple $\mathbf{r}$, which vanishes where the water has no depth. As the camera moves or tilts, $\mathbf{x}_b$ slides against the surface (parallax), and the animated term makes the bottom shimmer (refraction). The bottom itself is the shelf with a soft drop-off, the deeper floor, the shadow of the land, and the colony on the shelf, the latter from the same cell and region textures as on land. Evaluating the bottom in the water shader makes the result independent of the coarse terrain mesh and needs no copy of the colour or depth buffer.

Depth-test lift. The terrain mesh is coarse, so near the shore it can lie slightly above the plane inside the painted sea. The plane's clip-space depth is therefore taken from the same vertex raised by Δ_y,

$$z_{\text{clip}} \leftarrow w_{\text{clip}}\ \frac{z'_{\text{clip}}}{w'_{\text{clip}}}, \qquad (\cdot)' = \text{projection of } \mathbf{p} + (0, \Delta_y, 0), \tag{46}$$

which changes only the depth comparison, not the position on screen: the water wins against terrain up to Δ_y above it, while higher land still occludes the sea behind it. At the coast the plane does not end in a cut-out; its opacity fades over a narrow band of the smooth land mask, and its colour there is foam.

Shore. Foam is the band $D < \tau_f(\mathbf{x}, t)$ with a slowly breathing, noise-perturbed limit τ_f, pierced by hashed irregular holes, plus hashed flecks whose density decays with D. Lakes skip the shelf, the surf and the depth steps.

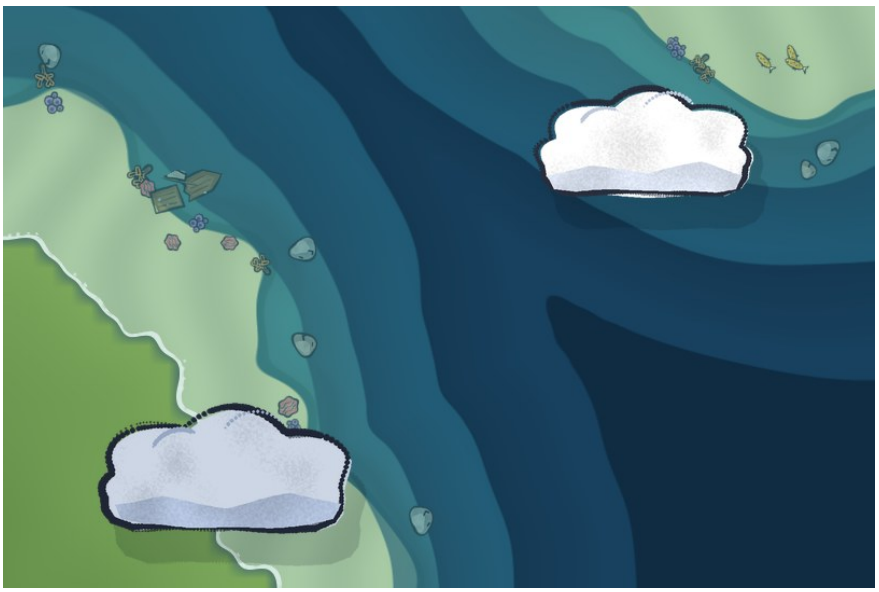


Figure 24: Water over the shelf and the deep, computed with the model of Section 6.4 (top view, without parallax).

6.5 Weather, night and light

Weather field. Each day the fronts of Section 5.2 are rasterised into a low-resolution four-channel texture (rain and storm, snow, fog, heat) with the corrections of Eq. (32) applied per texel; all channels fade towards the edge of the map, so that no bank of weather is cut by the border. Rain is drawn as hashed slanted dashes over darker ground, snowfall as hashed dots with a slow sway, heat as a warm tint. Fog is not a uniform haze but separate banks: with a drifting smooth noise $n \in [0,1]$ and the local fog amount a,

$$u = a\,\big(u_0 + (1-u_0)\,n\big), \qquad \alpha_{\text{fog}} = \alpha_1\,\text{step}_{\text{aa}}(u,u_1) + \alpha_2\,\text{step}_{\text{aa}}(u,u_2), \qquad u_0 < u_1 < u_2, \quad (47)$$

which leaves clear air wherever $u < u_1$; because $u_0 < u_1$, even a full-strength front ($a = 1$) keeps gaps where the noise is low (Fig. 22, bottom right).

Night. Night is a band that sweeps the map from east to west,

$$\mathcal{N}(x,t) = 1 - \text{smoothstep}\big(d_1, d_2,\ \big|\text{frac}(x/W + t/T_n) - \tfrac{1}{2}\big|\big), \qquad 0 < d_1 < d_2 < \tfrac{1}{2}, \qquad (48)$$

with x the horizontal map coordinate and T_n its period. Land, water and vegetation are blended towards a night tint by $\mathcal{N}$. The clock t here advances with simulated time and stops when the simulation is paused; the period T_n is a presentation constant that spans many simulated days, so the band is an ambience cue and not a calendar (Section 9).

Lights of settlements. Settlements are placed once per world: at the port and airport of a region, at a capital on the most habitable open cell near its centroid, and at a number of towns that grows with population; never in forest, on ice or on rock. Settlement s at $\mathbf{x}_s$ contributes a Gaussian bump, and the density map is their maximum,

$$\Lambda(\mathbf{x}) = \max_s\ \lambda_s \exp\!\big(-\|\mathbf{x} - \mathbf{x}_s\|^2 / r_s^2\big). \qquad (49)$$

The shader turns the density into a point process: a fine tile $\mathbf{j}$ shows a point of light iff its hash exceeds $1 - k_\Lambda\,\Lambda\,\iota_i$, where $\iota_i = \max(0, 1 - k_f f_i)^{3/2}$ is the light level of region i. Lights are therefore dense in the centre of a town and sparse at its fringe, thin out as the region is covered and go out before it is fully held; they are visible only where $\mathcal{N} > 0$.

7 Performance engineering

7.1 Methodology

Timings were taken with the C# code compiled for the .NET (Mono) runtime and run outside the engine after a warm-up pass. The browser build compiles the same code ahead of time to WebAssembly (IL2CPP), which is typically faster, so the figures are conservative.

7.2 Time-sliced generation

Executed in one call, generating a world on the larger grid took several hundred milliseconds in a single frame, a clearly visible freeze (Fig. 25a). The iterator of Section 3.11 is consumed under a fixed time budget per frame, so the same work spreads over a dozen or so frames while the loading animation continues (Fig. 25b). Map painting uses the same scheme.

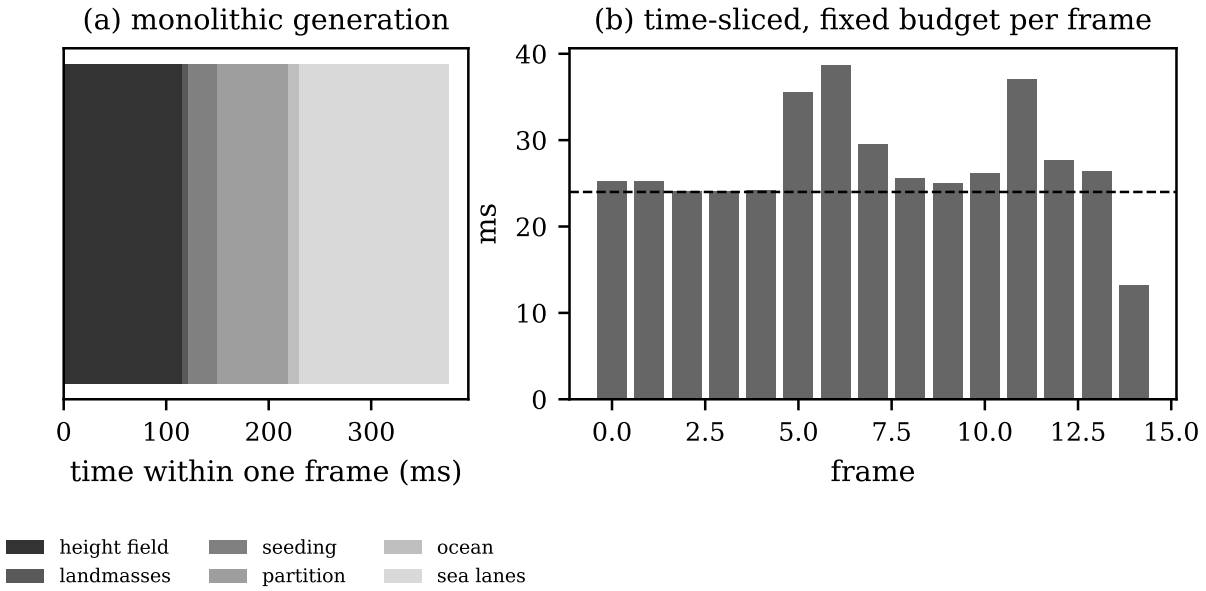


Figure 25: Generation on the larger grid. (a) Phase times when executed in a single frame. (b) Work per frame with time slicing.

7.3 Linear-time selection

Replacing the full sort of Section 3.4 by quickselect reduced the cost of finding the sea level by more than an order of magnitude on the larger grid while returning the identical value. It removed the longest uninterruptible step of the pipeline.

7.4 Tiled texture uploads

Uploading a whole data texture whenever a region is reached causes stalls at high simulation speed, when several regions may be reached within a second. Dynamic textures are therefore managed as grids of tiles. A change marks only the tiles covered by its bounding box (Fig. 26), and each frame at most a fixed number of dirty tiles is copied into a small staging texture and transferred into place with a GPU-side copy. This bounds the per-frame upload cost regardless of how much changed (Algorithm 7). Where copies between textures are unsupported, a full upload is performed at a limited rate.

Algorithm 7: Tiled upload of a dynamic texture, once per frame

Input: CPU copy of the texture, queue Q of dirty tiles, per-frame limit T

for $n \leftarrow 1$ **to** T ***while*** $Q \neq \emptyset$ **do**
    $t \leftarrow$ dequeue(Q); copy the pixels of t into the staging texture;
    upload the staging texture; GPU-copy it into the rectangle of t;

7.5 Cost of each subsystem

Table 1 summarises the asymptotic cost of the main computations, with N cells, N_R regions, n cells in one region, n_F fronts, M pixels of the base map and P pixels on screen.

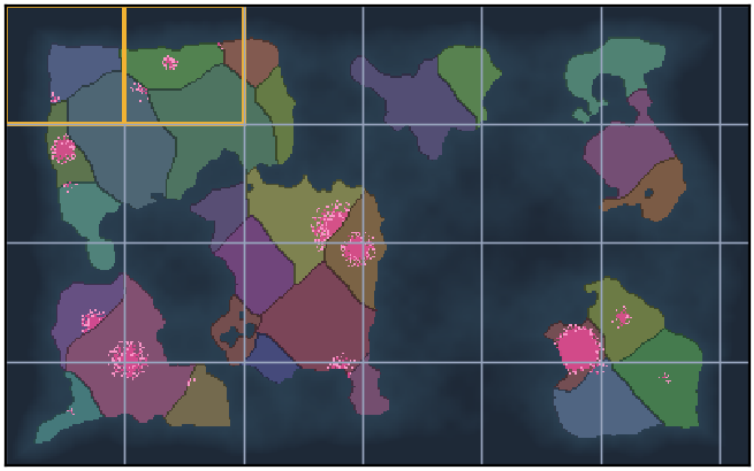


Figure 26: The tile grid of the cell texture. When a region is reached only its tiles (highlighted) are uploaded, a few per frame.

Table 1: Asymptotic cost and frequency of the main computations.

Computation	Cost	When
Noise, shaping, sea level (selection)	$O(N)$	once per world
Region partition (multi-source Dijkstra)	$O(N \log N)$	once per world
Maritime routing (A* per lane)	$O(N \log N)$ per lane, worst case	once per world
Distance to land, lake flood fill	$O(N)$	once per world
Base map painting with the vote of Algorithm 6	$O(M)$	once per world, time-sliced
Growth order of a region (Algorithm 4)	$O(n \log n)$	once per reached region
Rules of one simulated day	$O(N_R + \text{links})$	every simulated day
Environment of one day (Algorithm 5)	$O\big(n_F\,(N_R + N_\ell)\big)$, N_ℓ lattice texels	every simulated day
Region row upload	$O(N_R)$	every frame
Land, water and vegetation shading	$O(P)$ with $O(1)$ texture reads per pixel	every frame

7.6 Further measures

Off-screen vehicles and carriers are culled before drawing. Background traffic grows with the square root of the number of regions rather than linearly. Interface rows, effects, vehicles and transient icons are pooled to avoid garbage collection, and interface texts are rebuilt only when their value changes. Measured on the larger grid, one simulated day of rules costs a few hundredths of a millisecond on average, a negligible share of the frame budget even at accelerated speed. Optimisation effort was therefore concentrated on rendering, texture traffic and generation, which dominate the frame time.

7.7 Delivery size

For browser delivery the size of the downloaded build matters as much as runtime speed, and it is dominated by assets rather than code. An automated editor step applies the following measures and produces a size report after each build, listing content by kind and the largest

assets:

- **Textures** are stored with crunched block compression at the resolution at which they are actually displayed; mipmaps are kept only where an image is shown at many scales, and no CPU-side copies are retained.
- **Audio** is transcoded to the browser's native compressed format at a reduced sample rate, in mono, and trimmed to the sections that are actually played (intro and loop).
- **Fonts** are baked into compact signed-distance-field atlases containing only the glyphs in use, and the source font files are excluded from the build.
- **Code** is reduced by managed-code and engine stripping and size-oriented ahead-of-time compilation, and the rendering pipeline is configured without features the game does not use (high dynamic range, multisampling, shadows, depth and colour copies), with unused shader variants stripped.
- **Always-included content** (resource folders, splash images) is audited, so that nothing ships unless the build references it.

8 Validation

8.1 Determinism

Generation is a pure function of the seed, and a session is a pure function of the seed and of the sequence of player decisions with their times. Determinism is checked by hashing the generated grids and paths and comparing them across implementations (monolithic and time-sliced) and runs.

8.2 Balance with simulated players

A headless harness plays the real rules with scripted agents of increasing skill. The agents differ in how often they collect rewards, how often and how cleverly they buy upgrades (at random or following a plan), whether they act against laboratories and how often they use special actions. Each agent plays many generated worlds per difficulty level, and the harness reports win rates, session lengths and the state of the countermeasure at the end. The results confirm the intended properties: inactive play never wins, any active play usually wins at the standard difficulty, success at the higher difficulty depends clearly on activity and planning, and session lengths stay within the intended range.

Agents. An agent is a tuple $(p_c, \Theta, \lambda_L, \lambda_A, \mathcal{P})$: the probability p_c of collecting a reward before it expires, the interval Θ between purchase decisions, the daily rates λ_L and λ_A of acting against a laboratory and of using the special action, and a purchase policy $\mathcal{P}$ (uniform over affordable upgrades, or the same restricted to upgrades that do not raise visibility while the colony is still undetected). A probability over an interval is converted to a per-day probability so that the total is preserved: a reward that lives L days is collected on each day with probability $1 - (1 - p_c)^{1/L}$. Five agents span the range from inaction to attentive play, in the spirit of procedural personas [20].

Estimator. For a configuration (difficulty, organism variant, agent) the harness plays n worlds with fixed seeds, so that two configurations are compared on the same worlds. The win rate $\hat{p} = k/n$ is reported with the standard error $\sqrt{\hat{p}(1-\hat{p})/n}$; differences smaller than about two

standard errors are treated as noise. Session length is summarised by its median and by the first and last deciles over the won sessions, and each loss by the day it occurred and the world coverage reached, which distinguishes close losses from hopeless ones.

Sensitivity. Every tunable constant can be overridden on the command line of the harness. The balance is explored by one-at-a-time sweeps of a constant followed by a full re-run; the response of the win rate to the speed of the countermeasure is steep (roughly a sigmoid in the constant), which is why that constant is adjusted in small steps and always re-measured on all variants. The weather has a stream of its own (Section 5.2) and the growth order draws nothing from the stream of the rules (Section 4.4), so both can be varied and the balance re-measured on the same worlds; for the growth order the result is the same within noise, as expected from its weak coupling with the rules.

8.3 Statistics of the environment

The same harness runs the environment alone for several simulated years and tabulates, per climate, the share of region-days under each kind of weather. The weights of Eq. (31) and the corrections of Eq. (32) are accepted when three conditions hold: snow dominates precipitation over cold land, it never occurs over hot land, and every climate is under some weather for a comparable share of the time, large enough for the weather factor of Eq. (33) to matter.

8.4 Offline evaluation of procedural shading

Shaders cannot be unit-tested in the usual sense, and a visual defect is only seen in the running application. Two tools shorten that loop. First, the CPU painter of Section 6.1 is executed outside the engine: the engine's mathematics, vector and colour types are replaced by functional equivalents, the painting coroutine is driven until its first interaction with the user interface, and the pixel buffers are extracted by reflection; this yields the real base map of any seed as an image. Second, the fragment-shader mathematics of Sections 6.2 to 6.4 is transcribed to an array language and evaluated on that image, using the real threshold fields of Algorithm 4 dumped from the rules. The transcription uses the same hash and noise functions and estimates screen-space derivatives by finite differences, so that edges are as crisp as on the GPU. The figures of Section 6 were produced this way. The method does not replace testing in the engine (it cannot reveal compilation errors, precision differences or perspective effects), but it exposes errors of design, scale and frequency early.

8.5 Runtime

Generation, simulation and frame times are measured with the same harness and with the engine profiler, as described in Section 7. Time slicing keeps every frame close to the budget during generation, and the cost of the rules is negligible compared with rendering.

9 Discussion and limitations

The model favours legibility over epidemiological realism. Within a region the dynamics are deterministic and logistic, without recovery or spatial structure; spatial heterogeneity inside a region exists only in the visual model of Section 4.4, which is decoupled from the rules by construction. Transmission is evaluated per link and per day, so the waiting time for a jump is geometric, and correlations between simultaneous jumps are not modelled. The rubber band

of Eq. (24) deliberately trades strict monotonicity of difficulty for close contests; the simulated players show that it does not invert the ordering of skill. Runtime figures were measured outside the browser runtime and are therefore indicative. The environment is deliberately schematic: fronts are discs with a prescribed drift, not solutions of a flow model, and the two hemispheres differ only by a phase. The ground state is local (no run-off, no shading by relief). The night band is decoupled from the calendar: one sweep spans many simulated days, because a simulated day is a rule step that lasts a fraction of a second on screen. The water has no free surface; depth is a function of the distance to land, which is plausible near coasts and arbitrary in the open sea. The offline evaluation of Section 8.4 reproduces the shading mathematics but not the engine, so every change still requires a run on the target platform.

10 Conclusion

A single seed drives a deterministic pipeline of classical algorithms: value noise and fBm, order statistics, breadth-first labelling, farthest-point sampling, a noisy multi-source Dijkstra partition, union–find and weighted A*. It produces varied, connected and playable worlds that are delivered without frame stalls by time slicing. On top of it, a compact metapopulation model with logistic growth, stochastic link transmission, endogenous detection and an adaptive countermeasure yields sessions whose difficulty can be tuned and verified automatically with simulated players. A threshold-field visual model turns one number per region into an organic, creeping colony that the GPU draws at negligible cost; deriving the field from a multi-source geodesic arrival time and equalising it by rank gives each organism variant a recognisable way of spreading while making the displayed area an exact function of the simulated coverage. A light environment model (hemispheric seasons, a point process of weather fronts, relaxation dynamics for wetness, snow and dryness) feeds both the rules and the picture, and analytic, anti-aliased procedural shading renders land, water, weather and light in one consistent hand-drawn style without screen-space copies. Together these choices meet the goals of replayability, legible rules and fluid browser performance.

References

[1] K. Perlin. An image synthesizer. *ACM SIGGRAPH Computer Graphics*, 19(3):287–296, 1985.

[2] A. Lagae, S. Lefebvre, R. Cook, T. DeRose, G. Drettakis, D. S. Ebert, J. P. Lewis, K. Perlin and M. Zwicker. A survey of procedural noise functions. *Computer Graphics Forum*, 29(8):2579–2600, 2010.

[3] B. B. Mandelbrot. *The Fractal Geometry of Nature*. W. H. Freeman, 1982.

[4] D. S. Ebert, F. K. Musgrave, D. Peachey, K. Perlin and S. Worley. *Texturing and Modeling: A Procedural Approach*, 3rd ed. Morgan Kaufmann, 2003.

[5] C. A. R. Hoare. Algorithm 65: Find. *Communications of the ACM*, 4(7):321–322, 1961.

[6] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein. *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.

[7] T. F. Gonzalez. Clustering to minimize the maximum intercluster distance. *Theoretical Computer Science*, 38:293–306, 1985.

[8] Y. Eldar, M. Lindenbaum, M. Porat and Y. Y. Zeevi. The farthest point strategy for progressive image sampling. *IEEE Transactions on Image Processing*, 6(9):1305–1315, 1997.

[9] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.

[10] G. Voronoi. Nouvelles applications des paramètres continus à la théorie des formes quadratiques. *Journal für die reine und angewandte Mathematik*, 133:97–178, 1908.

[11] R. E. Tarjan. Efficiency of a good but not linear set union algorithm. *Journal of the ACM*, 22(2):215–225, 1975.

[12] P. E. Hart, N. J. Nilsson and B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

[13] G. M. Chaikin. An algorithm for high-speed curve generation. *Computer Graphics and Image Processing*, 3(4):346–349, 1974.

[14] W. O. Kermack and A. G. McKendrick. A contribution to the mathematical theory of epidemics. *Proceedings of the Royal Society of London A*, 115:700–721, 1927.

[15] P.-F. Verhulst. Notice sur la loi que la population suit dans son accroissement. *Correspondance mathématique et physique*, 10:113–121, 1838.

[16] J. A. Sethian. A fast marching level set method for monotonically advancing fronts. *Proceedings of the National Academy of Sciences*, 93(4):1591–1595, 1996.

[17] C. de Boor. *A Practical Guide to Splines*. Springer, 1978.

[18] L. Williams. Pyramidal parametrics. *ACM SIGGRAPH Computer Graphics*, 17(3):1–11, 1983.

[19] T. Akenine-Möller, E. Haines, N. Hoffman, A. Pesce, M. Iwanicki and S. Hillaire. *Real-Time Rendering*, 4th ed. CRC Press, 2018.

[20] C. Holmgård, A. Liapis, J. Togelius and G. N. Yannakakis. Evolving personas for player decision modeling. In *IEEE Conference on Computational Intelligence and Games*, 2014.

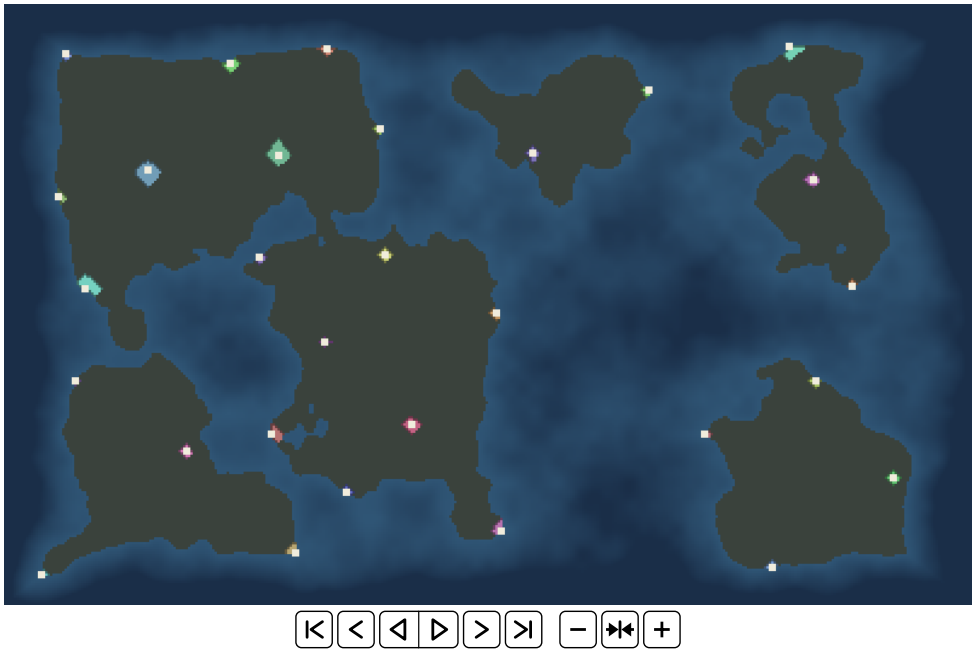
A Symbols

Some letters are reused with a different meaning in different sections, following the conventions of each field; the section is given where this happens.

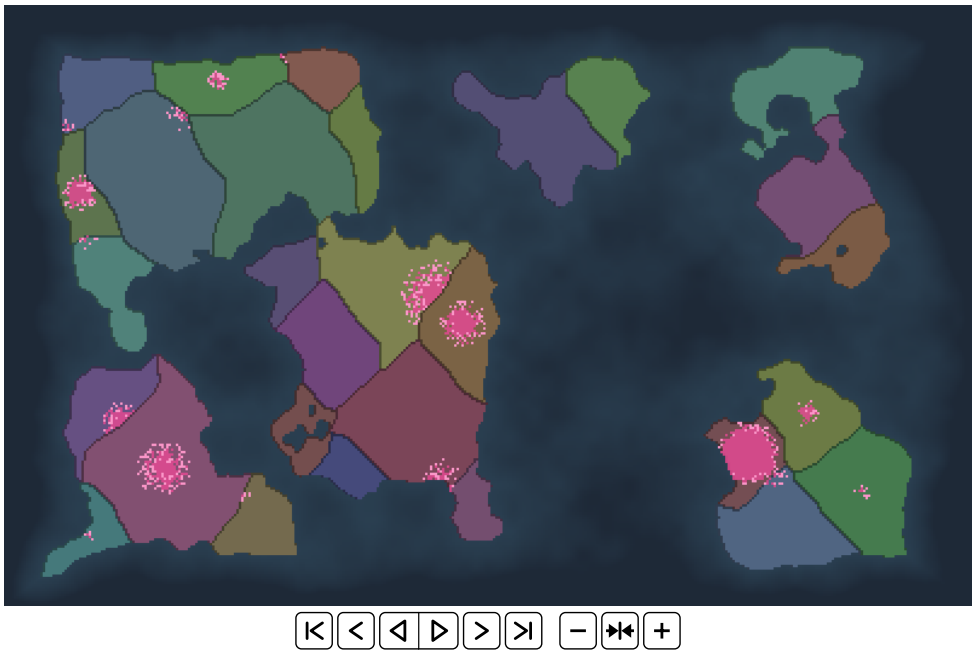
Symbol	Meaning
W, H, N	grid width, height, number of cells (Section 3)
$N(x,y), F(x,y)$	value noise, fractional Brownian motion
h, θ, λ	height, sea level, target land share
N_R	number of regions
$w(q), w(c \to c')$	step cost of the region partition, of the growth order
$\varsigma, \omega, \hat{h}$	distance to shore, coastal penalty, A* heuristic (Section 3.10)
ℓ	latitude, 0 at the equator and 1 at the poles
I_i, P_i, f_i	infected, population, coverage of region i
W (in Section 4)	world coverage
ρ	global pace
b_i, S_i, G_i, B_i	growth rate; seasonal, weather and temporary factors
$p_{ab}, \tau, \ell_{\text{type}}$	transmission probability, base rate, leak factor
ϑ_c, v_i, v_0	cell threshold, region visibility, its floor
$\tilde{h}, h_r, h_p, h_v, h_w$	normalised height; habit: relief, patchiness, veins, wind
ζ, T_0, T, ϱ	vein indicator, arrival times, start-time factor of a seed
$r_c, \mathcal{A}(f)$	rank fraction of a cell, share drawn as colonised
A, C, E	suspicion, countermeasure, rubber-band factor
$\mathcal{H}$	wealth-weighted healthy share
π_i	paranoia of region i
ν (in Section 4.5)	colony noise
Y, ψ, ψ_h	year length, season phase, phase in hemisphere h
$\sigma, \sigma_h, \varepsilon_i$	seasonal amplitude, temperature term, climate sign
Ω_k, a_k	season weights, season affinity
$F = (\mathbf{x}, \mathbf{u}, R, \varkappa, p, t_{\text{end}})$	weather front: centre, drift, radius, kind, strength, end
$c_j(\ell, k), \xi_j$	climatological weight and bias of weather kind j
$a_i, G^0_\varkappa, \Theta_\varkappa$	weather amount; base effect and taste for a kind
$\mathsf{w}, \mathsf{s}, \mathsf{d}, \chi$	wetness, snow, dryness of the ground; coldness
B, V_r, Δ_V	vote kernel, vote of region r, vote margin
D, Σ, Γ	distance to land, shelf width, water cover
$\mathbf{x}_b, z_b, n_r$	bottom point seen, bottom depth, refraction factor
$\mathcal{N}, \Lambda, \iota_i$	night factor, settlement density, light level of region i

B Supplementary animations

The animations below are embedded in the PDF and play in viewers that support JavaScript animation, such as Adobe Acrobat Reader (click to play); other viewers show the first frame. The same animations are provided as separate GIF files.



Animation S1. Multi-source Dijkstra growth of the regions with the noisy cost of Eq. (9).



Animation S2. A simulated session from early spread to saturation, drawn with the threshold model.